

SYSTEM INSTRUCTION: McTell Parametric CNC Joinery & SVG Generation Engine

Objective: Generate parametric manufacturing-ready SVG files for a six-sided, CNC-joinery shadowbox with internal Glazing Stops and a screwed-on back panel.

I. Strategic Compliance & Conflict Resolution

You are directed to treat the **"Design and Coding Principles"** and **"McTell CNC Shadowbox Specs"** as one unified document. Prior to the commitment of any SVG data, you must execute a comprehensive internal audit to ensure absolute adherence to every requirement within both documents. If a logical conflict arises or a requirement cannot be met, you must abort the process and provide a detailed Error Message; heuristic "guessing" is strictly prohibited. Any structural or geometric ambiguities must be presented to the user for explicit resolution before proceeding.

II. Code Generation & Filesystem Architecture

Your objective is to generate executable code for terminal deployment. This script must automate the export of SVG files to the following directory:
/Users/joelsilverman/Desktop/2026 Files/26-005 Gemini-created CNC Box Creator/Gemini McTell CNC Plans/. The system must implement incremental versioning logic, automatically detecting existing project subdirectories to create a new, uniquely named folder (e.g., "McTell SVGs v1", "McTell SVGs v2") for each successive run. This mechanism is required to maintain a rigorous organizational hierarchy and prevent the accidental overwriting of existing manufacturing assets.

III. Initialization & Variable Mapping

Before the generation of SVG geometry, you must output a structured, numbered list of all required variables for user specification. This initialization phase allows the user to provide values in a streamlined "number-to-value" format (e.g., 1. [value], 2. [value]). You will then map these inputs to the corresponding parameters within the script logic to finalize the manufacturing instructions.

IV. Geometric Audit Table

Before generating any code, you must produce a "Geometric Audit" table documenting the mating dimensions for every joint to ensure perfect mathematical alignment between parts. This audit must explicitly verify that tenon lengths match mortise depths and finger widths match slot widths across all interacting components. Do not proceed until this logic is validated and presented for review.

V. Joinery Method Selection

1. Initialization Prompt

At the immediate launch of every session, before performing any calculations, you must present the following menu exactly as written and wait for user input:

"Please select a joinery method:

Available Methods:

(a) BLIND BOX JOINT (Blind mortises and tenons, with a "skin" hiding the joint)

(b) HALF-ROUND BOX JOINT (Fingers will be flush with mating surface (if desired, you can manually smooth the edges of the fingers with a $\frac{1}{8}$ " small pilot roundover bit to make all edges look like interlocking Chiclets)"

2. Variable Mapping

* If the user selects (a), set the global variable ACTIVE_JOINERY_METHOD = BLIND_BOX_JOINT.

* If the user selects (b), set the global variable ACTIVE_JOINERY_METHOD = HALF_ROUND_BOX_JOINT.

3. Global Enforcement

The selected ACTIVE_JOINERY_METHOD applies to all jointed connections in the project (Front-to-Rail AND Rail-to-Rail) unless a part specification explicitly declares a different method. Explicit per-part joinery declarations in the CNC Shadowbox Specs take precedence over ACTIVE_JOINERY_METHOD for that specific connection only.

VI. Inside-Up Machining Rule

All SVG geometry must be generated from the perspective of the part's inside face facing up toward the CNC spindle. This ensures all internal pockets, rabbets, and motor mounts are machined on the correct face without requiring a manual flip of the stock.

VII. Virtual Assembly Match Report

Prior to writing any files, you must perform a "Virtual Assembly" check and output a Match Report for every jointed edge. This report must verify the complementarity of mating phases and confirm that the 0.010 inch glue gap is applied exclusively to female socket faces.

Design and Coding Principles for Coding Parametric CNC Box Generation Software

I. Geometric Core & Coordinate System

1. Single Source of Truth (The Volume)

The project is defined strictly as a virtual 3D volume (Width $\times$ Height $\times$ Depth). All 2D vectors must be derived mathematically from this master volume. No "free-floating" 2D geometry is permitted.

2. Coordinate System Orthodoxy

- Canvas: Fixed 48"x48" Artboard for all outputs.
- Global Origin: (0,0) is defined as the bottom-left corner of the artboard.
- Part Anchor: The geometry for every part must be anchored exactly 4.0 inches (X) and 4.0 inches (Y) from the Global Origin.

3. Unit Discipline & Dimensional Provenance

- Internal Units: All internal logic and math must use Millimeters (mm) to ensure precision.
- Explicit Conversion: Silent conversions are banned. A dedicated function (e.g., `convert_to_mm(value, unit)`) must be used for all inputs, logging both the input and output values to the console (e.g., `convert_to_mm(14.5, "inches")` → 368.3mm).
- Provenance Logging: Critical derived dimensions must log their "provenance formula" to the console for auditability.

II. Topological Logic

4. Hybrid Joinery Topology

The system must explicitly distinguish between two edge types:

- Jointed Edges: Mating connections (Front-to-Side) using finger joints.

- Back Panel Abutment Edges: Flat profile cuts (Back-to-Side) designed to accept the rabbeted Back Panel (screwed, not jointed). *Previously referred to as Profile Edges.*

5a . Edge Identity & Phase Contract

Every jointed edge must be an object declaring: ID, Orientation, Start-Phase (START_WITH_FINGER or START_WITH_GAP), and Parity. Mating edges are forbidden from generation unless they possess mathematically complementary phases.

5b. GAP AVOIDANCE

Alternate Corner Parity (The "Finger-Socket" Handshake): For every corner, one rail must provide the "Finger" (solid material) and the other must provide the "Socket" (gap). You cannot have Socket-Socket or Finger-Finger at a corner. Every corner has exactly one solid "block" overlapping a "void," guaranteeing no gaps.

- **Top Rail:**
 - **Left End:** Starts with a **FINGER**.
 - **Right End:** Ends with a **SOCKET**.
- **Right Rail:**
 - **Top End:** Starts with a **FINGER** (fills Top Rail's socket).
 - **Bottom End:** Ends with a **SOCKET**.
- **Bottom Rail:**
 - **Right End:** Starts with a **FINGER** (fills Right Rail's socket).
 - **Left End:** Ends with a **SOCKET**.
- **Left Rail:**
 - **Bottom End:** Starts with a **FINGER** (fills Bottom Rail's socket).
 - **Top End:** Ends with a **SOCKET** (accepts Top Rail's finger).

Verify this parity with no gaps for every mating pair before generating geometry.

6. Explicit Male/Female Assignment Table

A static, immutable lookup table defines the "gender" of every mating pair to prevent circular logic (e.g., **FRONT_BEZEL** → Always Female).

7. Handedness & Mirroring Guardrail

No 2D geometric mirroring (e.g., scale(-1, 1)) on finished vectors. Right and Left parts must be generated independently from the Volume. Mirrored pairs must undergo validation to ensure joinery phases remain complementary.

III. Manufacturing & CAM Strategy

8. Machining Orientation (Inside-Up)

All SVGs are generated assuming the Inside Face is Up on the CNC bed to enable pocketing without flipping the stock.

9. Layer-Based CAM Strategy

Output SVGs must strictly adhere to the following layer/color mapping for Carbide Create:

- Black: Outer Profile (Cut Through)
- Blue: Pocket Regions (Rabbets/Dados)
- Red: Inner Cutouts (Window)
- Magenta: Text/Metadata (No Cut)

10. Dogbone Orthodoxy

Every internal corner must receive a tool-radius fillet.

- Selection Criteria: T-bone fillets shall be used where the fillet would otherwise be visible on an exterior face. Dogbone fillets shall be used on concealed interior corners.
- Square internal corners are forbidden.

11. Manufacturability Rules Engine

A centralized engine must enforce physical constraints and abort generation if violated. Constraints include:

- Finger Width: Must be $\geq$ Tool Diameter.
- Wall Thickness: Must be $\geq$ Minimum Material Safety.
- Screw Clearance: Back panel holes must have sufficient edge distance.
- Pocket Depth: Minimum pocket depth relative to tool flute length.
- Aspect Ratio: Maximum pocket aspect ratio to prevent tool deflection.
- Spacing: Minimum distance between adjacent pockets.

12. Tolerance Offset (Female-Only, Axis-Aware)

A 0.010" glue gap shall be applied strictly to Female Socket interior faces only along the joint normal axis. It is prohibited on Male fingers, Profile edges, and Pocket perimeters.

IV. Feature Logic

13. "Glazing Stop" Size

The dimensions of the internal glass pocket will be calculated by automatically subtracting the dimensions of the Glazing Stops from the internal box volume.

Use the formula in Section 30.B ($\text{WINDOW_CUTOUT_WIDTH} + 2 * \text{GLASS_OVERLAP}$) to explicitly define the Glass Pocket size. The Glazing Stops will be cut to fit whatever space remains.

14. Blind Joint "Skin" Thickness

When a "blind joint" method is specified with a "skin" that covers the joint to reach a corner, the skin shall be 3mm.

15. Parametric Multi-Method Joinery

- **Enumeration:** A `JOINERY_METHOD` parameter must control execution.
- **Supported Types:**
 - `HALF_ROUND_BOX_JOINT` (Woodsmith Spec)
 - `BLIND_BOX_JOINT` (Woodsmith "Coverlet" Spec)
- **Logic Source:** The geometric definitions for these joints are **explicitly defined in Appendix A**. The software must not "infer" or "learn" these shapes from external media; it must strictly implement the algorithmic logic provided in Appendix A.
- **Constraint:** Selecting an unimplemented method must trigger an immediate error stub.
- **Optimization:** Tabs must automatically recalculate quantity to remain optimal for the 1/4" end mill regardless of box size, unless specified otherwise by the joinery design.

16. The Squareness Imperative

- **Constraint:** All mating edges must be generated at exactly 90.000° to their adjacent edges.
- **Enforcement:** Any computed angle deviating from 90° by more than 0.001° shall abort generation with a critical error.

17. Algorithmic Hierarchy

The logic will reference the Active Joinery Strategy for baseline geometry, but CNC Best Practices and Physical Manufacturability (Principles 10 & 11) override the reference algorithm whenever a conflict arises.

V. Output, Verification & Versioning

18. UI-Friendly Separation

One unique SVG per part with metadata (Part Name, Dimensions) visually centered above the part geometry.

19. Topology, Closure & Joint Validation

Before writing files, the system must run a "Virtual Assembly" check including Global Closure and a Per-Joint Match Report (PASS/FAIL status for every mating pair).

20. Visual Verification (Cut List)

A text-based "Cut List" summarizing all parts and dimensions must be printed to the console.

21. Golden-Reference Regression Testing

The script must include an automated test suite containing canonical scenarios (Standard, Thin Stock, Deep Box) that run on execution to prevent regression.

22. Incremental Versioning

You will create "one shot" code" that I will paste into a Terminal and it will export a series of SVGs inside subfolders inside this folder: /Users/joelsilverman/Desktop/2026 Files/26-005 Gemini-created CNC Box Creator/Gemini McTell CNC Plans/ Inside, you will create an incrementally named series of folders (i.e. McTell SVGs v1, McTell SVGs v2, etc) and inside you will save SVGs with matching incremental version numbers. The script must automatically detect the project directory and create a new, incremented subdirectory for every run (e.g., v1, v2) to ensure no file is ever overwritten.

The script must automatically detect the project directory and create a new, incremented subdirectory for every run (e.g., v1, v2) to ensure no file is ever overwritten.

23. Ambiguities

Every aspect of "McTell Shadowbox Specs" (the design specs) and "Design and Coding Principles" (this document) must be adhered to. Before committing any SVG to a file, run through every requirement in both documents to make sure you have met every requirement. Stop and give an Error Message if you find a conflict or if you cannot meet every requirement

of both documents in each SVG. You are not allowed to ever "guess" the answer to an ambiguity. You must ask the user to handle any ambiguities.

24. Variable Initialization & Parametric Defaults

Upon receiving the Joinery Method selection, you must immediately proceed to Variable Initialization. Present the following prompt exactly as written and wait for the user's numbered response.

"Please approve variables. Respond with numbered values (e.g., '1. 14.8, 14. 40.0'), or simply respond 'Accept Defaults'.

- a. Measured material thickness (Default 15mm / 0.59 in)
- b. Tool diameter for primary end mill (Default 0.25" / 6.35mm)
- c. Tool diameter for detail end mill (Default 0.125" / 3.175mm)
- d. Glue gap tolerance (Default 0.010" / 0.254mm)
- e. Tab peak thickness (Default 3.00mm)
- f. Tab base width (Default 12.7mm / 0.5")
- g. Target finger width for box joints (Default 15mm / 0.59 in)
- h. Blind joint skin thickness (Default 3.0 mm)
- i. Back panel rabbet width (Default 0.625" / 15.875mm)
- j. Back panel rabbet depth (Default 0.3" / 7.62mm)
- k. Stepper motor cutout tray? (Y/N, Default: Y) — If Y, select size: NEMA-08 (20×20mm), NEMA-11 (28×28mm), NEMA-14 (35×35mm), or NEMA-17 (42×42mm). Default: NEMA-17.
- l. 12. NEMA motor pocket depth (Default 0.25" / 6.35mm) — Skipped if Variable #11 = N.
- M. 10/32 Screw pilot hole diameter for back panel screw hole (Default 5.2 mm)
- n. Total Box Width (Default 40.0")
- o. Total Box Height (Default 40.0")
- p. Box Depth (Default 4.0")

q. Bezel Width (Default 1.59")

r. Glass Overlap (Default 0.84")

s.. Glazing Setback (Default 0.5")

t. Motor Position X (Distance from Left Inside Edge) (Default: $0.4155 \times \text{PHOTOGRAPH_WIDTH}$, approximately 16.13" for a 40" box with 15mm rails).

u. Motor Cable Offset (Distance from Motor Pocket) (Default 3.0")

25. Labels

Labels must be rendered in Josefin Sans Semibold at 80 pt, black color ([#000000](#)). Subtext (such as dimensions) shall use Josefin Sans Regular at 60 pt, black color. If Josefin Sans is not available on the host system, the script shall fall back to Arial Bold (labels) and Arial Regular (subtext) with a console warning: "Josefin Sans not found; using Arial fallback."

Labels must not overlap with SVG geometry and must maintain a minimum 1" (25.4mm) clearance from any cut path in the document. Label anchor points shall be calculated dynamically based on part bounding box plus the clearance offset.

Font Source (if installation required): <https://fonts.google.com/specimen/Josefin+Sans>

26. Centered Objects

27. Centered Objects

The following parts shall be centered on the 48"×48" artboard:

- TOP_RAIL, BOTTOM_RAIL, LEFT_RAIL, RIGHT_RAIL (individually centered)
- GLAZING_STOPS (centered as a collection)

The following parts shall be anchored per Principle 2 (4.0" from origin):

- FRONT_BEZEL
- BACK_PANEL

In the event of a conflict between Principle 2 and Principle 27, Principle 27 takes precedence for the parts listed above.

28. Onion-Skin Tab & Cutout Frame Geometry

Dimensional Specification: Every part perimeter will be held in place during the profile cut by a specific "Onion-Skin" tabbing strategy involving a secondary outer cut line.

A. The Cutout Frame (Green Layer)

- **Definition:** The script must generate a rectangular vector frame (color: `#00FF00`, layer: `GREEN`) around every individual part.
- **Offset Logic:**
 - **Flat Edges:** The frame must be offset from the part edge by exactly **6.35 mm (0.25")**. This gap defines the "Tab Length."
 - **Jointed Edges (Protruding Fingers):** To prevent collisions, the frame must be offset by `STOCK_THICKNESS + 6.35 mm`.
- **Machining:** This Green Frame is the *final* operation (Step 6). It is cut as a through-cut to release the block of material containing the part from the main stock.

B. Trapezoidal Tab Markers (Magenta Layer)

- **Purpose:** Visual markers indicating where the CAM operator must place 3D tabs connecting the Part (Black Line) to the Cutout Frame (Green Line).
- **Geometric Profile:** Tabs are **Trapezoidal** to ensure a robust connection to both the workpiece and the waste frame.
 - **Tip Width (Part Connection):** Fixed at **3.175 mm (0.125")**. This ensures sufficient material contact with the finished part.
 - **Base Width (Frame Connection):** Fixed at **12.7 mm (0.5")**.
 - **Orientation:** The narrow tip touches the Part (Black Line); the wide base touches the Cutout Frame (Green Line).
- **Unit Provenance:** The 0.5" base and 0.125" tip must undergo explicit conversion via `convert_to_mm()`.

C. Z-Depth Specification

- **CAM Requirement:** Tabs must be defined in the CAM software to be milled down, leaving **3.175 mm (0.125")** of material remaining on the table (Z-Height). This thickness provides holding strength while remaining thin enough for manual severing.

D. Placement Rules & Conflict Resolution

- **Dogbone Conflict:** Where a tab might intersect a delicate joint feature (like a blind socket), the logic must shift the tab to a solid surface (e.g., a "Finger" rather than a "Socket").
- **Part-Specific Population:**

i. Rails (Top/Bottom/Left/Right): Limited to exactly **two tabs** per component. These must be placed exclusively on the non-jointed (flat) edge to preserve the joinery fingers on the opposite side.

ii. Front Bezel: Tabs must be placed on all four sides of the outer perimeter.

- *Parity Rule:* Tabs must align with **Fingers** (solid wood), not Sockets (air). The script must target the **2nd Finger** from each corner (Indices 2 and Count-3) to ensure solid contact.

iii. Back Panel: Tabs shall be centered on each of the four sides of the perimeter.

iv. Glazing Stops: Each strip shall have two tabs per long edge (top and bottom), centered along the length. No tabs on short edges.

29. Carbide Create CAM Cut Instructions

FUNCTION **Generate_CAM_Text_File()**:

a. Define Filename:

Set **filename** = "x CAM_Instructions.txt" (Prefix "x" ensures file sorts last in Finder).

b. Open File for Writing:

- Header: Write Project Name, Generation Timestamp, and Active Joinery Method.
- Cut List: Write a text summary of all parts (Name, Qty, W x H).

c. Generate Color Map & Depth Specification:

Constraint: Do NOT include any "Tabbing Guide" or reference to tab locations.

Constraint: Explicitly calculate and list the Cut Depth for every color found in the SVGs.

WRITE "COLOR MAP & MACHINING SPECS:"

- BLACK (Outer Profile):
 - **Operation:** Contour / Outside Cut
 - **Tool:** #201 0.25" End Mill
 - **Depth:** **STOCK_THICKNESS** + **0.25mm** (Cut Through into spoilboard)
- RED (Inner Cutouts & Holes):
 - **Operation:** Pocket (for Windows) or Helical Drill (for Holes)

- **Tool:** #201 for large windows; #102 0.125" for holes < 6.35mm
- **Depth:** STOCK_THICKNESS + 0.25mm (Cut Through)
- **BLUE (Pockets & Rabbets):**
 - **Operation:** Pocket / Area Clearance
 - **Tool:** #201 0.25" End Mill
 - **Specific Depths:**
 - Back Panel Rabbet: Depth = BACK_PANEL_RABBET_DEPTH
 - Motor Pocket (If Enabled): Depth = NEMA_POCKET_DEPTH (Default 0.25")
 - Blind Joints (If Active): Depth = STOCK_THICKNESS - BLIND_SKIN
 - Corner Reliefs (If Active): Depth = STOCK_THICKNESS + 0.25mm (Through)
- **GREEN (Onion-Skin Cutout Frame):**
 - **Operation:** Contour / Outside Cut
 - **Tool:** #201 0.25" End Mill
 - **Depth:** STOCK_THICKNESS + 0.25mm (Cut Through)

d. Specify Machining Sequence for User to Create in CAM Software:

Make this easy to read with double hard returns between paragraphs

Step 1: (Tool #102) Helical mill all 5mm pilot holes (Back Panel) and 3.4mm mount holes (Bottom Rail).

Step 2: (Tool Change -> #201) — only if MOTOR_POCKET_ENABLED = TRUE

Step 3: Pocket all BLUE regions (Rabbets, Motor Recess, Joinery Pockets).

Step 4: Profile/Cut-Through all RED regions (Windows, Shaft Holes).

Step 5: Profile/Cut-Through all BLACK regions (Part Perimeters). **ENSURE TABS ARE SET.**

Step 6: Profile/Cut-Through all **GREEN** regions (Onion-Skin Cutout Frame). This releases the block.

30. Parametric Geometry Drivers (Formulas & Relationships)

To ensure the design scales safely, the following formulas must be applied in place of hardcoded numbers.

A. Global Dimensions

* $TOTAL_WIDTH = \text{User Input \#14 (Default 40.0")}$

* $TOTAL_HEIGHT = \text{User Input \#15 (Default 40.0")}$

* $BOX_DEPTH = \text{User Input \#16 (Default 4.0")} \rightarrow \text{Drives Rail Height.}$

B. Face & Bezel Logic

* $WINDOW_CUTOUT_WIDTH = TOTAL_WIDTH - (2 * \text{User Input \#17 [Bezel Width]})$

* $WINDOW_CUTOUT_HEIGHT = TOTAL_HEIGHT - (2 * \text{User Input \#17 [Bezel Width]})$

* $GLAZING_WIDTH = WINDOW_CUTOUT_WIDTH + (2 * \text{User Input \#18 [Glass Overlap]})$

* $GLAZING_HEIGHT = WINDOW_CUTOUT_HEIGHT + (2 * \text{User Input \#18 [Glass Overlap]})$

* $GLAZING_Z_POS = \text{User Input \#19 [Glazing Setback] from front face.}$

* $PHOTOGRAPH_WIDTH = TOTAL_WIDTH - (2 \times USER_DEFINED_THICKNESS) *$

$PHOTOGRAPH_HEIGHT = TOTAL_HEIGHT - (2 \times USER_DEFINED_THICKNESS) *$

$MOTOR_POCKET_X = \text{User Input \#20 [Motor Position X] (Default: } 0.4155 \times \text{ PHOTOGRAPH_WIDTH)}$

C. Internal Component Logic

* $MOTOR_POCKET_ENABLED = \text{User Input \#11 (Y/N, Default: Y)}$

* $MOTOR_POCKET_SIZE = NEMA_FRAME_SIZES[\text{User Input \#11 selection}]$
(Default: 42.0mm for NEMA-17)

* $MOTOR_POCKET_X = \text{User Input \#20 [Motor Position X] (Default: } 0.4155 \times \text{ PHOTOGRAPH_WIDTH)}$

* $CABLE_HOLE_X = MOTOR_POCKET_X - \text{User Input \#21 [Motor Cable Offset]}$

* $GLAZING_STOP_STOCK_WIDTH = \text{User Input \#1 [Material Thickness]}$

D. Centering Logic

* All Edge Drilling (Pilot Holes) $\rightarrow$ Center at $\text{User Input \#1 [Material Thickness]} / 2.0.$

* Back Panel Pilot Holes -> Recalculate "PanelWidth / 3" based on actual BACK_PANEL_WIDTH.

31. Code Generation Policy for Software Development Phase

a. VERSIONING

Each iteration creates a new version folder: McTell Gemini SVGs v{N}/

Python script named mctell_CNC_v{N}.py

SVG files named {PART_NAME}_v{N}.svg

Version history block at top of script accumulates one-line summaries (minimum 15 words per comment) describing each revision

b. UNITS

Set units to mm. Convert user-provided variables given in inches to millimeters.

c. LABEL FORMATTING

(Magenta #FF00FF, No Cut)

Part name: Josefin Sans Semibold, 80pt (NOT 80 SVG units — actual 80 points)

Dimensions: Josefin Sans Regular, 60pt (NOT 60 SVG units — actual 60 points)

Hard return (separate <tspan>) between label and dimensions

Round numbers display without decimals: i.e. 40" not 40.00"

Fractional numbers display with 2 decimals: i.e. 36.82"

SVG font-size is in user units (inches in this case).

d. RAIL ORIENTATIONS IN SVG FILES

TOP_RAIL: rotation=180° (teeth face DOWN — default, toward viewer/front)

However DO NOT ROTATE THE TOP_RAIL label and dimensions

BOTTOM_RAIL: rotation=180° (teeth face UP)

LEFT_RAIL: rotation=-90° (teeth face RIGHT)

RIGHT_RAIL: rotation=90° (teeth face LEFT)

e. RAIL STYLING

Fill: #FF97F5 (light pink)

Stroke: #000000 (black)

Stroke-width: 1pt = 0.3528mm in SVG mm units

:

f. BEZEL/PANEL STYLING

Fill: none (transparent)

Stroke: Black (#000000) for profile cuts, Red (#FF0000) for inner profiles, Blue (#0000FF) for pockets

Stroke-width: 0.5mm (current) or as specified

g. ARTBOARD SETTINGS

Canvas Size: Every SVG must be generated on a fixed 48" x 48" artboard.

h. SVG ARTBOARD

Use explicit inches: <svg width="48in" height="48in" viewBox="0 0 48 48" ...> (Assuming user units are inches. If units are mm, convert 48" to 1219.2mm).

Placement: Ensure the part geometry is positioned safely within this 48" square (do not let it sit at negative coordinates).

i. OUTPUT PATH

/Users/joelsilverman/Desktop/2026 Files/26-005 Gemini-created CNC Box Creator/Gemini McTell CNC Plans/McTell SVGs v{N}/

j. COMBINED FILE

Make a file named COMBINED.SVG that has all files generated in this process into one layered file.

32. IMPLEMENTING JOINERY ON ALL MATED EDGES

You must implement joinery on ALL mating edges of the Rails, not just the ends.

- End-Joints: Connect Rail-to-Rail (Horizontal Rails mate into Vertical Rails).
- Face-Joints: Connect every Rail to the Front Bezel. The 'Long Edge' of every rail (Top, Bottom, Left, Right) must have a corresponding finger/socket pattern that matches the Bezel's perimeter.
- Do not leave any rail edge as a simple straight line unless it is the back edge receiving the screwed-on panel. Every Rail must have three active joinery edges: End 1, End 2, and the Face Edge.

33. Joinery Parity Inheritance CONSTRAINT

When generating **mating pairs** (e.g., Bezel-to-Rail, Rail-to-Rail), you must strictly follow this **Master/Slave** logic to ensure perfect alignment:

1. Define the MASTER part:

- For Face Joints: The **FRONT_BEZEL** is the Master.
- For End Joints: The **VERTICAL RAILS** are the Master.

2. Calculate ONCE:

- `master_finger_count = compute_layout(MASTER_DIMENSION)`
- `master_finger_width = MASTER_DIMENSION / master_finger_count`

3. Inherit for SLAVE part:

- **NEVER** recalculate `count` or `width` based on the Slave part's physical length (which may include protrusions or offsets).
- **ALWAYS** force the Slave part to use `master_finger_count` and `master_finger_width` directly.
- *Example:* `Rail_Top_Finger_Count = Bezel_Bottom_Finger_Count` (Exact Integer Match).

FAILURE CONDITION: If `Slave_Finger_Count != Master_Finger_Count`, the script is invalid.

34. BALTIC BIRCH LAYOUT GUIDE

Requirement: Upon generating the project files, the software must generate one additional SVG file in the output folder named `Plywood_Layout_Guide_v[Version].svg`.

Functional Specifications:

1. **Scale & Units:** The SVG must be drawn in **Inches** at 1:1 scale.
2. **Sheet Format:** Render two (2) 60" x 60" Baltic Birch sheets side-by-side with a 5" gap.
3. **Dynamic Geometry:** Do not use hardcoded sizes. Dimensions for the Bezel, Rails, Back Panel, and Glazing Strips must be derived directly from the user's `CONFIG` variables (converted to inches).
4. **Layout Logic:**
 - **Sheet 1:** Front Bezel (Bottom Left), Top Rail, 4x Glazing Strips.
 - **Sheet 2:** Back Panel (Bottom Left), Right Rail, Left Rail, Bottom Rail.
 - **Margins:** Maintain a strict 2.0" safety margin from the bottom and left edges of the plywood sheets.
 - **Spacing:** Automatically calculate even spacing for the rails/strips into the remaining area to the right of the main panels.
5. **Visual Style (The "Guide" Look):**
 - **Exploded Objects:** No groups (`<g>`). All elements must be root-level.
 - **Part Fill:** Light Blue (`#d1e8ff`).
 - **Profile Stroke:** Black (`#000000`) at 1pt width (0.0139").
 - **Cut Box:** A Pure Red (`#FF0000`) rectangle offset 0.3" from the profile, representing the visual tab/cut boundary.
 - **Tabs:** Magenta (`#FF00FF`) triangular markers on the profile line.
 - **Fonts:** "Josefin Sans" (Regular). Size 80pt for main labels, 40pt for glazing strips.

Code Implementation for Baltic Birch Layout Guide

Paste this function **inside** your CNC_Shadowbox_Generator .py script (e.g., before generate_cam_instructions).

Then, modify the run_generation function to call it.

Python

```
# =====
# NEW FUNCTION: PASTE THIS BEFORE generate_cam_instructions
# =====

def generate_plywood_layout_svg(output_dir, version_str):
    print("GENERATING: Plywood Layout Guide")

    # 1. Retrieve & Convert Dimensions to Inches
    # CONFIG stores mm, layout needs inches
    def to_in(mm): return mm / 25.4

    sheet_w, sheet_h = 60.0, 60.0
    safe_margin = 2.0
    gap_between_sheets = 5.0

    # Actual Part Dimensions
    panel_w = to_in(CONFIG['TOTAL_WIDTH'])
    panel_h = to_in(CONFIG['TOTAL_HEIGHT'])
    rail_len = to_in(CONFIG['TOTAL_WIDTH']) # Approx max length for visualization
    rail_h = to_in(CONFIG['TOTAL_HEIGHT']) # For vertical rails
    rail_depth = to_in(CONFIG['BOX_DEPTH'])

    # Glazing Strips (Oversized in app, so we use max dimension + 5 inches)
    strip_len = max(panel_w, panel_h) + 5.0
    strip_w = 0.75 # Standard width for strips

    # Styles
    col_sheet = "#e3d2b4"
    col_part_fill = "#d1e8ff"
    col_profile_stroke = "#000000"
    col_cut_box = "#FF0000"
    col_tab = "#FF00FF"
    stroke_width = 0.013888 # 1pt

    font_family = "JosefinSans-Regular, Josefin Sans, sans-serif"
    sz_std_inch = 80/72.0
    sz_glz_inch = 40/72.0

    tab_base = 0.5
    tab_h = 0.3

    # --- HELPERS ---

    def get_abs_tabs(px, py, w, h):
        tabs = []
        def make_tri(cx, cy, direction):
            acx, acy = px + cx, py + cy
            if direction == 'UP': pts = f'{acx-tab_base/2},{acy} {acx+tab_base/2},{acy} {acx},{acy-tab_h}'
```

```

elif direction == 'DOWN': pts = f'{acx-tab_base/2},{acy}{acx+tab_base/2},{acy}{acx},{acy+tab_h}'
elif direction == 'LEFT': pts = f'{acx},{acy-tab_base/2}{acx},{acy+tab_base/2}{acx-tab_h},{acy}'
elif direction == 'RIGHT': pts = f'{acx},{acy-tab_base/2}{acx},{acy+tab_base/2}{acx+tab_h},{acy}'
return f'<polygon points="{pts}" fill="{col_tab}" stroke="none" opacity="0.9"/>'

if w > 5:
    for x_pct in [0.20, 0.80]:
        tabs.append(make_tri(w*x_pct, 0, 'UP'))
        tabs.append(make_tri(w*x_pct, h, 'DOWN'))
    else:
        tabs.append(make_tri(w/2, 0, 'UP'))
        tabs.append(make_tri(w/2, h, 'DOWN'))

if h > 5:
    for y_pct in [0.20, 0.80]:
        tabs.append(make_tri(0, h*y_pct, 'LEFT'))
        tabs.append(make_tri(w, h*y_pct, 'RIGHT'))
return "\n".join(tabs)

def create_exploded_part(label, x, y, w, h, is_waste=False):
    elements = []
    # Fill
    elements.append(f'<rect x="{f(x)}" y="{f(y)}" width="{f(w)}" height="{f(h)}" fill="{col_part_fill}" stroke="none" />')
    # Profile
    elements.append(f'<rect x="{f(x)}" y="{f(y)}" width="{f(w)}" height="{f(h)}" fill="none" stroke="{col_profile_stroke}"
stroke-width="{stroke_width}" />')
    # Red Cut Box
    rx, ry = x - tab_h, y - tab_h
    rw, rh = w + (tab_h * 2), h + (tab_h * 2)
    elements.append(f'<rect x="{f(rx)}" y="{f(ry)}" width="{f(rw)}" height="{f(rh)}" fill="none" stroke="{col_cut_box}"
stroke-width="{stroke_width}" />')
    # Tabs
    elements.append(get_abs_tabs(x, y, w, h))
    # Label
    cx, cy = x + w/2, y + h/2
    f_sz = sz_glz_inch if "GLAZING" in label else sz_std_inch
    elements.append(f'<text x="{f(cx)}" y="{f(cy)}" font-family="{font_family}" font-size="{f(f_sz)}" fill="black"
text-anchor="middle" dominant-baseline="middle" transform="rotate(-90 {f(cx)} {f(cy)})">{label}</text>')
    # Waste
    if is_waste:
        wsz = 36.0 # Approx 36 inches for waste area, could be calculated but 36 is safe visual
        wx, wy = x + (w - wsz)/2, y + (h - wsz)/2
        elements.append(f'<rect x="{f(wx)}" y="{f(wy)}" width="{f(wsz)}" height="{f(wsz)}" fill="#ffcccc" stroke="black"
stroke-width="{stroke_width}" stroke-dasharray="0.2, 0.2" opacity="0.5"/>')
    return "\n".join(elements)

def get_y(part_h):
    return sheet_h - safe_margin - part_h

# --- SVG CONTENT ---
svg_content = [f'<?xml version="1.0" encoding="UTF-8"?>']
<svg width="{sheet_w*2 + gap_between_sheets + 5}in" height="{sheet_h + 10}in"
viewBox="-2 -10 {sheet_w*2 + gap_between_sheets + 5} {sheet_h + 10}"
xmlns="http://www.w3.org/2000/svg">

```

```

<rect x="0" y="0" width="{sheet_w}" height="{sheet_h}" fill="{col_sheet}" />
<rect x="{sheet_w + gap_between_sheets}" y="0" width="{sheet_w}" height="{sheet_h}" fill="{col_sheet}" />

<rect x="{safe_margin}" y="{safe_margin}" width="{sheet_w - 2*safe_margin}" height="{sheet_h - 2*safe_margin}"
fill="none" stroke="gray" stroke-dasharray="0.5, 0.5" stroke-width="{stroke_width}" />
<rect x="{sheet_w + gap_between_sheets + safe_margin}" y="{safe_margin}" width="{sheet_w - 2*safe_margin}"
height="{sheet_h - 2*safe_margin}" fill="none" stroke="gray" stroke-dasharray="0.5, 0.5" stroke-width="{stroke_width}"
/>

<text x="{sheet_w/2}" y="-7" font-family="{font_family}" font-size="2" font-weight="bold"
text-anchor="middle">Baltic Birch: SHEET 1</text>
<text x="{sheet_w + gap_between_sheets + sheet_w/2}" y="-7" font-family="{font_family}" font-size="2"
font-weight="bold" text-anchor="middle">Baltic Birch: SHEET 2</text>
""

# --- SHEET 1 LOGIC ---
# Bezel
bezel_x = safe_margin
bezel_y = get_y(panel_h)
svg_content.append(create_exploded_part("FRONT BEZEL", bezel_x, bezel_y, panel_w, panel_h, is_waste=True))

start_x = bezel_x + panel_w
avail_width = (sheet_w - safe_margin) - start_x

# Parts: Top Rail + 4 Glazing Strips
# Note: Using max rail length for visualization safety
parts_s1 = [("TOP RAIL", rail_depth, rail_len)] + [(f"GLAZING {i}", strip_w, strip_len) for i in range(1, 5)]

total_parts_width = sum([p[1] for p in parts_s1])
gap_size = (avail_width - total_parts_width) / (len(parts_s1) + 1)
if gap_size < 0.7: gap_size = 0.7

cur_x = start_x + gap_size
for label, pw, ph in parts_s1:
    svg_content.append(create_exploded_part(label, cur_x, get_y(ph), pw, ph))
    cur_x += pw + gap_size

# --- SHEET 2 LOGIC ---
# Back Panel
s2_origin = sheet_w + gap_between_sheets
back_x = s2_origin + safe_margin
back_y = get_y(panel_h)
svg_content.append(create_exploded_part("BACK PANEL", back_x, back_y, panel_w, panel_h))

start_x = back_x + panel_w
avail_width = (s2_origin + sheet_w - safe_margin) - start_x

# Parts: Right, Left, Bottom Rails
# Using rail_h for vertical rails (Left/Right) if different, but usually visualization uses longest dimension for packing
parts_s2 = [("RIGHT RAIL", rail_depth, rail_h), ("LEFT RAIL", rail_depth, rail_h), ("BOTTOM RAIL", rail_depth, rail_len)]

total_parts_width = sum([p[1] for p in parts_s2])
gap_size = (avail_width - total_parts_width) / (len(parts_s2) + 1)
if gap_size < 0.7: gap_size = 0.7

```

```

cur_x = start_x + gap_size
for label, pw, ph in parts_s2:
    svg_content.append(create_exploded_part(label, cur_x, get_y(ph), pw, ph))
    cur_x += pw + gap_size

svg_content.append('</svg>')

# WRITE FILE
filename = f"Plywood_Layout_Guide_v{version_str}.svg"
with open(output_dir / filename, 'w') as fh:
    fh.write("\n".join(svg_content))
print(f" OK {filename}")

```

How to Integrate:

1. Paste the code above into `CNC_Shadowbox_Generator .py`.
2. Find the `run_generation` function (inside the `ShadowboxApp` class).
3. Add the call to `generate_plywood_layout_svg` **after** the `generate_cam_instructions` call, like this:

Python

```

# ... existing code ...
for fname, content in files.items():
    with open(final_out_dir / fname, 'w') as f:
        f.write(content)

generate_cam_instructions(final_out_dir, next_ver)

# --- NEW LINE HERE ---
generate_plywood_layout_svg(final_out_dir, next_ver)
# -----

messagebox.showinfo("Success", f"SVGs generated successfully! ..

```

APPENDICES

VI. Appendix A: Geometric Logic & Algorithmic Reference (This section overrides external "learning" requirements. The code must implement these specific geometric steps to ensure manufacturability.)

1. Global Parameters & Constants

Python

MATERIAL & TOOLING

```

t_mm = USER_DEFINED_THICKNESS (Default 15.0)      # Measured Thickness (Must be a
variable, not hardcoded)
tool_d_mm = 6.35      # Tool Diameter (1/4" End Mill)
tool_r = tool_d_mm / 2.0
fit_tolerance = USER_DEFINED_GLUE_GAP (Default 0.254mm)  # Glue gap (applied to
female pockets only)

# HALF-ROUND SPECS (Woodsmith Ep 002)
hr_protrusion = 2.0      # EXTRA length for fingers to allow post-process round-over
hr_roundover_r = 3.175  # The radius of the handheld router bit (1/8")

# BLIND BOX SPECS (Woodsmith "Coverlet") blind_skin = USER_DEFINED_BLIND_SKIN
(Default 3.0) # Thickness of the hiding wall (Coverlet) blind_pocket_depth = t_mm - blind_skin

# STEPPER MOTOR POCKET SPECS # Frame sizes per NEMA standard (User Input #11)
NEMA_FRAME_SIZES = {
    "NEMA-08": 20.0, # 20mm × 20mm frame
    "NEMA-11": 28.0, # 28mm × 28mm frame
    "NEMA-14": 35.0, # 35mm × 35mm frame

    "NEMA-17": 42.0 # 42mm × 42mm frame (Default)
}

# Mounting hole patterns per NEMA standard (center-to-center distance) NEMA_MOUNT_PATTERNS = {
"NEMA-08": 16.0, # 16mm square pattern "NEMA-11": 23.0, # 23mm square pattern "NEMA-14": 26.0, #
26mm square pattern "NEMA-17": 31.0 # 31mm square pattern (Default) } MOTOR_MOUNT_PATTERN
= NEMA_MOUNT_PATTERNS[USER_SELECTED_NEMA] # Set by User Input #11

```

2. Helper Logic

```

Python
FUNCTION ComputeFingerLayout(edge_len, target_width):
    # Ensure odd number of fingers for symmetry so pattern is centered
    count = round(edge_len / target_width)
    IF count is EVEN: count = count + 1
    actual_width = edge_len / count
    RETURN (count, actual_width)

FUNCTION ApplyTolerance(dim, type):
    IF type == "GROW": RETURN dim + fit_tolerance
    IF type == "SHRINK": RETURN dim - fit_tolerance

```

3. Logic: Half-Round Box Joint

Intent: Fingers join FLUSH with the mating surface. We are implementing a **"Corner Fill" strategy**: Place a *0.125"* square block (Positive Geometry) inside the corners of the socket. Internal material must be left on the 90° inside edges to allow the user to route the insides with a 1/8" round-over bit after CNC milling to match the CNC's rounded fingers with no gaps showing when mated.

Python

```
FUNCTION Generate_HalfRound_Joint(Part_A_Socket, Part_B_Finger):
```

```
    (n, w) = ComputeFingerLayout(edge_len, TARGET_FINGER_WIDTH)
```

```
    # --- PART A: SOCKETS (Female) ---
```

These are standard through-cuts, but they are undercut on 90° inside edges, requiring the user to manually route the insides with a 1/8" round-over bit after CNC milling to match the CNC's rounded fingers so no gaps show when mated.

```
    FOR each socket_band in Part_A:
```

```
        socket_w = ApplyTolerance(w, "GROW")
```

```
        # Geometry: Rectangle through full thickness
```

```
        draw_rect(width=socket_w, height=t_mm)
```

CRITICAL: NO T-BONES or Dogbones. Because the mating fingers will be rounded to match the tool radius, the natural rounded corners left by the CNC router in these sockets are desired. Do not add any fillet relief

```
    # --- PART B: FINGERS (Male) ---
```

These must be LONGER than the mating stock thickness.

```
    FOR each finger_band in Part_B:
```

```
        finger_w = ApplyTolerance(w, "SHRINK")
```

```
        # CRITICAL: Add Protrusion for Round-over
```

```
        finger_len = t_mm + hr_protrusion
```

```
        draw_rect(width=finger_w, height=finger_len)
```

```
        # Note: No dogbones on male fingers.
```

4. Logic: Blind Box Joint (The "Coverlet")

Intent: Pockets on Part A are blind (do not cut through). Fingers on Part B are pocketed on the INSIDE face to fit into Part A, creating a seamless exterior corner.

```
FUNCTION Generate_Blind_Joint(Part_A_Face, Part_B_Side):
```

```
    (n, w) = ComputeFingerLayout(edge_len, TARGET_FINGER_WIDTH)
```

```

# --- PART A: THE "COVERLET" (Female/Face) ---
# Concept: Blind Pockets. The "skin" remains on the OUTSIDE face.
FOR each socket_band in Part_A:
    socket_w = ApplyTolerance(w, "GROW")

    # OPERATION: POCKET (Blind)
    # Depth is CRITICAL. Do not cut through.
    # Depth = Material_Thickness - blind_skin
    set_cut_depth(t_mm - blind_skin)

    # GEOMETRY
    rect = draw_rect(width=socket_w, height=t_mm)

    # CRITICAL: "Hidden T-Bone"
    # Standard dogbones would ruin the visible skin.
    # Place the T-bone circle completely INSIDE the pocket boundaries.
    add_hidden_tbone(rect, r=tool_r)

# --- PART B: THE POCKETED FINGERS (Male/Side) ---
# Concept: Fingers must be shortened to fit in the blind pocket depth,
# AND pocketed on the INSIDE face to match the pocket thickness.

FOR each finger_band in Part_B:
    finger_w = ApplyTolerance(w, "SHRINK")

    # DIMENSION 1: Length Calculation
    # Finger length must match the depth of the pocket in Part A (minus glue gap).
    finger_len_geo = (t_mm - blind_skin) - 0.2mm

    # DIMENSION 2: The Inside Face Pocket (Z-Axis Inversion)
    # To strictly adhere to Principle VI (Inside-Up Machining), we remove material
    # from the INSIDE (Top) face. This leaves the remaining finger material
    # flush with the OUTSIDE (Bottom) face.

    # Step 1: Generate basic finger vector
    draw_rect(width=finger_w, height=finger_len_geo)

    # Step 2: Mark the "Lap" region for pocketing
    # Mill the entire surface of the finger down by 'blind_skin' thickness.
    # This creates a finger that is thinner than the stock, sitting flush
    # with the bottom (outside) of the rail.
    define_region_pocket(
        zone=entire_finger_area,
        depth=blind_skin
    )

```

)

5. Verification Protocol

Python

FUNCTION Run_Verification():

Mandatory Safety Step before full panel cut

1. Generate "Test Coupon" (Small corner sample of Part A + Part B).
2. Cut with fit_tolerance = 0.1mm.
3. Dry fit.
 - IF tight: Increase tolerance to 0.15mm.
 - IF loose: Decrease tolerance to 0.05mm.
4. Check Blind Joint Skin: Ensure 3mm skin is not transparent/fragile.

McTell CNC Shadowbox Specs

Creation Date: December 24, 2025

1. Project Overview

This project involves the design and fabrication of a **TOTAL_WIDTH** x **TOTAL_HEIGHT** shadowbox display case using **USER_DEFINED_THICKNESS** Baltic Birch plywood.

Construction Topology (6-Sided) Calculate SVGs based on **USER_DEFINED_THICKNESS**.
OVERALL BOX OUTSIDE DIMENSIONS ARE **TOTAL_WIDTH** x **TOTAL_HEIGHT**.

The **back panel** has a rabbeted edge defined by **BACK_PANEL_RABBET_WIDTH** and **BACK_PANEL_RABBET_DEPTH** so that it fits snugly into the back of the frame.

The **front panel** is glued in place with box-cut tabbed fingers. Also the front panel has a centered cutout reveal defined by **WINDOW_CUTOUT_WIDTH** for glazing to be dropped inside.

Glass Fastening method: The glass panel will be held in place with Fletcher points, covered with glued-in plywood Glazing Stops. CNC fabrication of the Glazing Stops will be limited to rough milling these parts to lengths suitable for miter-cutting later.

Actual glazing size is **GLAZING_WIDTH** x **GLAZING_HEIGHT**. Note that the glazing will be larger than the window cutout because the window cutout is a bezel.

The BOM and cut list will be as follows:

All wood parts to be cut from Baltic birch with actual dimensions **USER_DEFINED_THICKNESS** thick.

FRONT_BEZEL: $\text{TOTAL_WIDTH} \times \text{TOTAL_HEIGHT} \times \text{USER_DEFINED_THICKNESS}$ A window defined by **WINDOW_CUTOUT_WIDTH** x **WINDOW_CUTOUT_HEIGHT** will be cut out, centered inside the face, and the remaining bezel will form the structural front. All four edges are secured by **ACTIVE_JOINERY_METHOD**. This part will act as **Part A (Female/Coverlet)** as defined in Appendix A when a **blind box joint** mode is selected.

If a non-blind mode is selected (i.e. **Half-Round joint**), the Front Bezel becomes a **Standard Through-Socket Board (Part A)** with fingers visible on face, following the Half-Round logic in Appendix A.

TOP_RAIL: The **Visible Span** (Shoulder-to-Shoulder) is $(\text{TOTAL_WIDTH} - 2 \times \text{STOCK_THICKNESS})$. Join to **FRONT_BEZEL** using **ACTIVE_JOINERY_METHOD**. This part is

Part B (Male/Rabbit). Join to LEFT_RAIL and RIGHT_RAIL using **ACTIVE_JOINERY_METHOD**. This part is Part B (Finger).

LEFT_RAIL: Outer dimension is **TOTAL_HEIGHT** × **BOX_DEPTH** × **USER_DEFINED_THICKNESS**. Join to FRONT_BEZEL using **ACTIVE_JOINERY_METHOD**. This part is Part B (Male/Rabbit). Join to TOP_RAIL and BOTTOM_RAIL using **ACTIVE_JOINERY_METHOD**. This part is Part A (Socket).

RIGHT_RAIL: Outer dimension is **TOTAL_HEIGHT** × **BOX_DEPTH** × **USER_DEFINED_THICKNESS**. Join to FRONT_BEZEL using **ACTIVE_JOINERY_METHOD**. This part is Part B (Male/Rabbit). Join to TOP_RAIL and BOTTOM_RAIL using **ACTIVE_JOINERY_METHOD**. This part is Part A (Socket).

The **Left and Right Rails** run the full **TOTAL_HEIGHT** of the box. The Top and Bottom Rails run *between* them. The "Coverlet" (skin) is on the Left/Right rails.

BOTTOM_RAIL: The Visible Span (Shoulder-to-Shoulder) shall be defined as (**TOTAL_WIDTH** – 2 × **STOCK_THICKNESS**). Join to FRONT_BEZEL using **ACTIVE_JOINERY_METHOD**. This part is Part B (Male/Rabbit). Join to LEFT_RAIL and RIGHT_RAIL using **ACTIVE_JOINERY_METHOD**. This part is Part B (Finger).

Conditional Feature (User Input #11): If **MOTOR_POCKET_ENABLED** = True, generate the following motor pocket on BOTTOM_RAIL. If **MOTOR_POCKET_ENABLED** = False, skip this entire feature block.

At **MOTOR_POCKET_X** from the inside-left edge of the frame (measured along X), BOTTOM_RAIL has a **MOTOR_POCKET_SIZE** × **MOTOR_POCKET_SIZE** × **NEMA_POCKET_DEPTH** rectangular pocket cut (not a through-cut) for the selected NEMA stepper motor. Default for **MOTOR_POCKET_X** = 43.5% of **WINDOW_CUTOUT_WIDTH**, measured from the inside-left edge of the frame (i.e., **MOTOR_POCKET_X** = 0.435 × **WINDOW_CUTOUT_WIDTH**).

MOTOR_POCKET_SIZE is determined by User Input #11:

- NEMA-08: 20mm × 20mm
- NEMA-11: 28mm × 28mm
- NEMA-14: 35mm × 35mm
- NEMA-17: 42mm × 42mm (Default)

All four internal corners of this pocket shall receive dogbone fillets (radius = `tool_r`) per Principle 10, oriented inward to remain concealed beneath the motor mounting plate. A central 0.5" hole shall be bored through the center of this pocket for access to the motor's rear shaft. Mill four 3.4mm holes using helical interpolation in a square pattern centered on the pocket for M3 mounting screws. The mounting hole spacing is determined by **NEMA_MOUNT_PATTERN**, which varies by motor size:

- NEMA-08: 16.0mm × 16.0mm
- NEMA-11: 23.0mm × 23.0mm
- NEMA-14: 26.0mm × 26.0mm
- NEMA-17: 31.0mm × 31.0mm (Default)

Mill a 0.5" diameter pass-through hole at **CABLE_HOLE_X** (i.e., **MOTOR_POCKET_X** – **MOTOR_CABLE_OFFSET**) to route cables.

BACK_PANEL: Outer dimension is **TOTAL_WIDTH** × **TOTAL_HEIGHT** × **USER_DEFINED_THICKNESS**. Inside of the back panel there will be a centered photograph mounted on 1/4" foam core sized to fit within the rabbets. The back panel will have a rabbet cut defined by **BACK_PANEL_RABBET_WIDTH** (Default 0.625") and **BACK_PANEL_RABBET_DEPTH** (Default 0.3") cut around all four sides so that it fits over the back and can be screwed into place through the rabbeted portions. Drill pilot holes defined by **10_32_SCREW_DIA** (Default 5.2mm) to allow passage of 10/32 screws. These holes shall drill exactly through the bottom of the rabbet.

The Back Panel sits **on top** of the back of the rails (covering the rail ends completely). The "rabbet" cut creates a tongue that drops *inside* the frame to register it and then the Back Panel is screwed in place to secure it to the four rails.

There should be twelve total pilot holes drilled through the rabbet floor of **BACK_PANEL**, centered at **(BACK_PANEL_RABBET_WIDTH / 2.0)** from the panel's outer edges, placed as follows:

- **Corner Holes (4 total)**: One pilot hole at each corner, positioned **(BACK_PANEL_RABBET_WIDTH / 2.0)** from both adjacent edges.
- **Top Edge Mid-Holes (2 total)**: Two pilot holes along the top edge, positioned at **(TOTAL_WIDTH / 3)** and **(2 × TOTAL_WIDTH / 3)** from the left edge, centered at **(BACK_PANEL_RABBET_WIDTH / 2.0)** from the top edge.
- **Bottom Edge Mid-Holes (2 total)**: Two pilot holes along the bottom edge, positioned at **(TOTAL_WIDTH / 3)** and **(2 × TOTAL_WIDTH / 3)** from the left edge, centered at **(BACK_PANEL_RABBET_WIDTH / 2.0)** from the bottom edge.

- **Left Edge Mid-Holes (2 total):** Two pilot holes along the left edge, positioned at $(TOTAL_HEIGHT / 3)$ and $(2 \times TOTAL_HEIGHT / 3)$ from the top edge, centered at $(BACK_PANEL_RABBET_WIDTH / 2.0)$ from the left edge.
- **Right Edge Mid-Holes (2 total):** Two pilot holes along the right edge, positioned at $(TOTAL_HEIGHT / 3)$ and $(2 \times TOTAL_HEIGHT / 3)$ from the top edge, centered at $(BACK_PANEL_RABBET_WIDTH / 2.0)$ from the right edge.

F. GLAZING_STOPS Mill four plywood strips as follows:

- **GLAZING_STOP_HORIZONTAL (Qty 2):** Length = $(TOTAL_WIDTH + 5.0")$
- **GLAZING_STOP_VERTICAL (Qty 2):** Length = $(TOTAL_HEIGHT + 5.0")$

Each strip shall be milled to **USER_DEFINED_THICKNESS** wide. Eventually (manually) these will be miter-cut to fit inside the frame and ripped down to 0.3" thick to match the inside depth of the frame, then glued in to capture the glass. All four GLAZING_STOPS may be placed horizontally with 2" vertical separation on the same SVG.

Addendum: JOINERY PRINCIPLES

The following external resources serve as the academic source for the *intent* of the joinery, but the **geometric implementation is strictly defined in Appendix A**. In the event of a conflict between the video/article and Appendix A, **Appendix A takes precedence**.

Refer to Will Adams as a “Reference Library” of techniques regarding: Blind miter, pocketing logic, and dogbone constraints.

Source: https://github.com/WillAdams/Design_Into_3D

JOINERY PRINCIPLES FOR FULL BLIND BOX JOINT (also known as the Rabbeted Blind Box Joint)

The main source of truth for the method of joinery is this Woodsmith video:

CNC Basecamp Ep002: Joinery with a CNC

https://youtu.be/_In9McUHV9s?si=CPV-rCHAEYtyRpbl

06:36 to 07:48

1. Asymmetric "Male/Female" Part Definition

Unlike standard box joints where both boards are cut with identical patterns, this joint requires two distinct geometries. Part A (The Socket Board) acts as the drawer front and features blind pockets. Part B (The Finger Board) acts as the drawer side and features a rabbeted step. You cannot swap these parts; one acts as a "lid" (the front) and the other tucks behind it.

2. The "Coverlet" (Blind Socket Wall)

On Part A, the sockets must not cut all the way through the material. You must stop the cut short, leaving a thin wall of material (typically 1/8" to 1/4") on the outer face. Woodsmith calls this the "coverlet"—it is essentially a built-in veneer that hides the end grain of the mating board.

3. The "Tab" (Rabbeted Finger Base)

On Part B, you must machine a rabbet (a shoulder or step) along the entire mating edge. The depth of this rabbet must exactly match the thickness of the "coverlet" from Principle #2. This ensures that when assembled, the side board sits flush behind the front face, rather than creating a gap or a bulge.

4. Square Fingers, T-Bone Sockets

The fingers on Part B are generated as standard rectangles. However, because a round CNC bit cannot cut square corners inside the blind pockets of Part A, you must generate "T-bone" fillets at the inner corners of every socket. These small relief notches clear out the rounded corners left by the bit, allowing the square fingers to slide all the way in.

5. Z-Depth Correspondence

The length of the fingers on Part B must correspond to the depth of the pocket on Part A, not the full thickness of the material. If the front board is 0.75" thick but the pockets are only 0.5" deep, the fingers must be cut to 0.5" long. If they are longer, they will bottom out against the blind wall and prevent the joint from closing.

6. Glue Gap Clearance (Hydraulic Lock Prevention)

Apply a slight negative offset (shortening) to the finger dimensions. Because blind holes act like a piston in a cylinder, they trap air and glue at the bottom. You must make the fingers slightly shorter and thinner than the sockets to provide a small escape path for this pressure, otherwise, the joint will spring back open.

7. Single-Axis Visibility

Orient the "Coverlet" (Part A) to be the primary visible face. This joint is designed so that the front face looks like a solid, uncut piece of wood. The side view (Part B) will show a single vertical seam where the "Coverlet" meets the "Tab".

8. Pocketing vs. Profiling

The toolpath strategies differ for each half. Part A requires "Area Clearance" (pocketing) operations to excavate the blind sockets inside the board boundaries. Part B requires "Profile" (cut-out) operations to define the fingers, combined with a clearing pass to create the rabbeted shoulder.

9. Corner Structural Integrity

The pattern on Part A should begin and end with a Socket (a solid wall), rather than a finger gap. This ensures that the very top and bottom edges of the drawer front are supported by solid wood, rather than leaving the thin, fragile "coverlet" exposed at the corners where it could easily chip.

10. "Clean Internal Line" Geometry

Woodsmith specifies using T-bone fillets rather than standard "dogbones" for this joint. Align the fillet cut along the side wall of the socket rather than diagonally. This hides the relief notch along the edge of the finger, keeping the visible interior seam of the box clean and professional-looking.

JOINERY PRINCIPLES FOR HALF-ROUND BOX JOINT

The main source of truth for the method of joinery is this Woodsmith video:

CNC Basecamp Ep002: Joinery with a CNC

https://youtu.be/_In9McUHV9s?si=CPV-rCHAEYtyRpbl

Timecodes: 09:48 to 10:04

1. **Tool-Driven Geometry:** The entire joint geometry must be derived from the Cutter Radius (r). Unlike standard box joints where bit size only matters for kerf, here r dictates the shape of the joint. You must enforce that Finger Corner Radius = Socket Bottom Radius = r . Justification for this: Since a CNC router bit is round and spins, it cannot cut a square corner inside a slot; it always leaves a rounded corner matching the size of the bit. To make the joint fit without gaps, we must shape the "fingers" (the parts sticking out) to match that exact curve, rather than trying to force a square finger into a rounded slot.
2. **Male-Side Compensation:** Do not apply "dogbone" or "T-bone" fillets to the sockets (female parts). Instead, apply the geometric compensation to the fingers (male parts) by rounding their tips. This creates the "clean on the inside" look Woodsmith describes. Justification for this: Usually, to make CNC parts fit, makers cut ugly little "mouse ears" (dogbones) into the corners of the slots to clear out the waste. This method avoids that messy look. By rounding off the hidden tips of the fingers instead, we keep the visible parts of the joint looking clean, straight, and traditional.
3. **The "Half-Round" Condition:** Ideally, set the Finger Width (w) to exactly $2 \times r$ (or select a bit where diameter= w). Under this condition, the two corner radii of the finger meet in the center, creating a perfect semicircle (half-round) finger tip. Justification for this: This is the "Goldilocks" scenario for this joint. If the finger is exactly twice as wide as the radius of the cutter, the curves from the left and right corners will meet perfectly in the middle. This creates a smooth, perfect half-circle shape at the end of the finger, which is the most aesthetically pleasing version of this joint.
4. **Tangency & Continuity:** If the finger width is wider than $2r$, the finger tip must consist of a straight segment connected to two corner arcs. These arcs must be tangent (G1 continuous) to the side walls and the flat tip, preventing jerky machine motion or steps in the finish. Justification for this: If the finger is wide, it will have a flat spot on the very tip with curved corners on the sides. The transition from the flat part to the curved part needs to be perfectly smooth—like a highway on-ramp—rather than a sharp angle. If it isn't smooth, the machine might stutter, leaving ugly marks or "steps" on the wood.

5. Interference-Free Fit (Tolerance): You cannot cut the finger radius exactly at r if the socket is also cut at r (zero tolerance leads to binding). Apply a negative offset (glue gap) to the finger path, typically $-0.005"$ to $-0.010"$, to allow for wood expansion and glue. Justification for this: In the real world, if you make a plug exactly the same size as the hole, it will get stuck because there is no room for air, glue, or the wood fibers to swell. We need to make the finger a tiny fraction of an inch smaller (about the thickness of two sheets of paper) so the parts can slide together easily without having to use a hammer.
6. Z-Axis vs. XY-Plane: The "rounding" described is in the XY plane (the cutting path of the CNC). Ensure the logic modifies the 2D vector profile, not a 3D surface mesh. The CNC creates the shape by moving in X and Y; the Z-depth is simply the material thickness. Justification for this: We are not trying to sculpt a 3D dome or sphere on the end of the board. We are simply telling the machine to drive in a curved line on the flat surface. Think of this like drawing a curved shape on a piece of paper (2D), rather than molding a shape out of clay (3D).
7. Avoid "Full Round" External Edges: Per my instruction to ignore the "bony/full round" version: Apply the rounding only to the finger tips (the vectors at the end of the board). Do not generate paths that round over the long edges of the fingers (the face of the board). Justification for this: We are strictly focusing on making the joint fit together, which involves shaping the ends of the fingers that go *inside* the slots. We are *not* trying to round over the long, outside edges of the box for decoration. The face of the board should remain flat and sharp.
8. Socket Depth Clearance: The depth of the socket (cut into the end of the board) must match the Finger Length + Clearance. Ensure the socket is cut deep enough (or through) so the rounded finger tip bottoms out on the rounded socket floor, or passes through cleanly without hitting square corners at the bottom of a blind cut. Justification for this: If the slot isn't cut deep enough, the curved finger will hit the bottom before the joint is fully closed. It is like trying to park a car in a garage that is too short—the door won't close. We have to ensure the "garage" (socket) is deep enough to accommodate the entire "car" (finger).
9. Corner Case Handling: If the Finger Width $< 2 \times r$, the joint is impossible to cut (the bit will erode the finger entirely). The logic must validate that $w \geq 2r$. If $w > 2r$, the tip is "Flat-Round-Flat"; if $w = 2r$, the tip is "Semicircle". Justification for this: You cannot cut a very skinny finger with a very fat drill bit—the bit would just eat the entire finger, leaving nothing behind. We must make sure the wooden finger is at least as wide as the cutter. If it is wider, the tip will be flat with rounded corners; if it is the same width, it will be a perfect half-circle.

CNC Box Joint Mathematics for Parametric Manufacturing

The mathematical foundations of CNC box joinery rest on a fundamental constraint: round endmills cannot cut square inside corners. This geometric reality drives every formula in parametric joint design. The **half-round condition** where finger width equals twice the tool radius ($w = 2r$) creates a perfect semicircular finger tip that mates seamlessly with CNC-cut radiused sockets. Standard tolerances of **0.010" (0.254mm)** applied to male fingers provide assembly clearance and glue gaps, while **odd finger counts** ensure centered symmetry at box corners.

The half-round condition creates perfect semicircular finger geometry

The core insight from Woodsmith CNC Basecamp Episode 002 is that CNC routers leave inside corners with a radius exactly equal to the tool radius ($r = D/2$, where D is endmill diameter). Rather than fighting this limitation with dogbone reliefs, the half-round approach embraces it by rounding finger tips to match.

The critical formula is $w = 2 \times r$, where w is finger width and r is the roundover bit radius. When both sides of a finger are rounded over with radius r , the two quarter-round profiles meet at the centerline, forming a smooth semicircle that fits perfectly into the CNC-cut socket's radiused bottom. For a 1/4" endmill ($r = 0.125"$), the ideal finger width becomes 0.25".

This approach requires a secondary operation with a handheld router using a solid-pilot roundover bit (approximately 3/32" pilot). The finger width must be carefully calculated so the rounded profile terminates exactly at the finger's edge without leaving flat sections or creating undercuts.

Minimum finger width for structural integrity follows a different formula: $w \geq 2 \times D$ (twice the endmill diameter, or four times the radius). This prevents weakening the structure through excessively thin fingers. For a 1/4" endmill, minimum structural finger width is 0.5"—double what the half-round condition requires. Designers must balance aesthetic preference against mechanical strength.

Blind pocket depth depends on preserved "coverlet" skin thickness

Blind box joints (also called rabbeted blind box joints) hide the finger/socket interlock from the outer surfaces by preserving a thin skin of material. The fundamental pocket depth formula is:

pocket_depth = material_thickness – blind_skin

Where blind_skin typically ranges from **2-3mm (1/16"- 1/8")** for adequate hiding with structural integrity. The mating finger length must correspond precisely:

finger_length = pocket_depth – tolerance_gap

This tolerance gap (typically 0.005"-0.010") prevents fingers from bottoming out in pockets during assembly. Z-axis accuracy is critical—zero the spindle from the spoilboard rather than the stock surface to maintain consistent tongue heights across all parts.

T-bone and dogbone fillets within blind pockets use the **diagonal placement formula**: fillet center distance from corner = $\sqrt{(r^2/2)} \approx 0.707 \times r$. This positions the relief circle at 45° to the corner, minimizing material removal while ensuring the endmill can reach fully into corners. For near-invisible dogbones, position the fillet center **0.2mm shy of the corner**—the remaining material creates a friction fit that may hold without glue.

Tolerance mathematics determines whether joints assemble cleanly

The industry-standard total clearance is **0.010" (0.254mm)**, equivalent to **0.005" per side**. This value originated from practical CNC woodworking with Baltic birch plywood and MDF, where it produces "nice tight and glueable joints" according to experienced practitioners.

Three application strategies exist:

Strategy	Male Fingers	Female Sockets	Use Case
Female-only (most common)	Nominal	Enlarged by 0.010"	Plywood with fixed thickness
Male-only	Reduced by 0.010"	Nominal	When socket dimensions are critical
Split	Reduced by 0.005"	Enlarged by 0.005"	Maximum precision

The critical rule is that tolerances are **additive**: 0.005" male reduction plus 0.005" female enlargement equals 0.010" total gap. Never apply full tolerance to both sides—this creates 0.020" clearance and sloppy joints.

CAM software behavior varies importantly. In Vectric products, negative pocket allowance creates larger pockets (correct for female tolerance), while positive allowance shrinks them. Inside profile cuts invert this logic. Fusion 360's "Stock to Leave" parameter works differently—negative values enlarge the cut.

Wood movement adds complexity to tolerance calculations. The simplified rule is **1% dimensional change per 4% moisture content change** for flat-sawn lumber. A 30" quartersawn white oak panel experiencing 3% MC change moves approximately 0.162" (5/32"). Cross-grain joints in solid wood may require 0.010"–0.030" additional clearance beyond the standard glue gap.

Parametric finger count algorithms enforce odd numbers for symmetry

The Design_Into_3D repository reveals the core algorithm for finger count calculation:

10. $fjcount = (\text{ceil}(\text{Height} / \text{Stock_Thickness}) / 2) \times 2 + 1$

11. $fjsize = \text{Height} / fjcount$

This formula guarantees an odd number by: (1) calculating raw count from height and stock thickness, (2) ceiling and halving, (3) doubling and adding one. With five fingers (odd), you get a center finger with equal counts on each side (pattern: 2-1-2). With four fingers (even), no center element exists, creating asymmetric corners.

Phase complementarity between mating parts requires offset patterns. The "master" board starts with a finger (protrusion) at pattern position 0; the "slave" board starts with a gap (recess) at the same position. In OpenSCAD implementations, this manifests as `insideCuts()` versus `outsideCuts()` functions generating complementary patterns. Every finger on Board A aligns with a gap on Board B.

The minimum viable joint uses three fingers (two finger pairs)—fewer than this lacks mechanical interlock strength. For boxes with consistent visual appearance, all edges should have identical finger counts, though this requires careful width planning. The "easy way" noted in woodworking forums is making each dimension an odd multiple of your standard finger size.

Dogbone geometry follows tool radius compensation formulas

The standard dogbone radius formula is $\text{fillet_r} \geq 1.1 \times \text{tool_radius}$ —the 10% addition ensures clearance for CAM software tolerance issues and slight tool runout. For a 1/8" endmill (radius 0.0625"), minimum dogbone radius is 0.069".

Three placement strategies serve different aesthetic goals:

The **diagonal (standard) dogbone** centers the relief circle at 45° to the corner, calculated as distance = $r \times 0.7071$ from the theoretical corner point. This removes the least material while ensuring corner clearance.

The **T-bone** extends relief in one direction only, perpendicular to one edge. The result is a clean interior corner visible from inside the assembly, with the relief notch visible only from outside.

Minimal dogbones (FabLab RUC method) position the circle just tangent to the corner (0mm overlap) or 0.2-0.3mm shy, creating near-invisible reliefs. This technique produces friction-fit joints that may hold without glue, though they're tighter to assemble.

V-endmill pocket depth calculation enables blind miter joints

For 90° V-endmills used in blind miter work, pocket depth relates to tip radius by:

$$\text{pocket_depth} = \text{v_radius} / \tan(\text{angle}/2)$$

Since $\tan(45^\circ) = 1$ for a 90° V-bit, this simplifies to $\text{pocket_depth} = \text{v_radius}$. The V-endmill diameter must be less than or equal to half the stock thickness for full blind miters. Will Adams' Design_Into_3D documentation details a two-pass approach for thicker stock: one pass at bottom of stock, one at mid-thickness, with overlap occurring when cut depth exceeds $\text{Stock_Thickness}/2$.

Quarter blind miter joints combine V-cuts with box joint pockets. The available joint height equals Box_Height minus rabbet areas at top and bottom. Joint count = $\text{floor}(\text{available_height} / (2 \times \text{endmill_cutting_depth}))$, with individual joint height = $\text{available_height} / \text{joint_count}$.

Practical measurement protocol ensures manufacturing accuracy

Before any production run:

1. Measure actual endmill diameter with digital calipers—cheap bits vary ± 0.010 " from nominal

2. Measure actual stock thickness—plywood varies from nominal by 1/32" or more
3. Account for spindle runout (quality spindles: 0.002", cheap spindles: 0.004"+); effective kerf increase = $2 \times \text{runout}$
4. Cut test rectangles at different allowances (0.000", 0.005", 0.010", 0.015") and number mating pairs

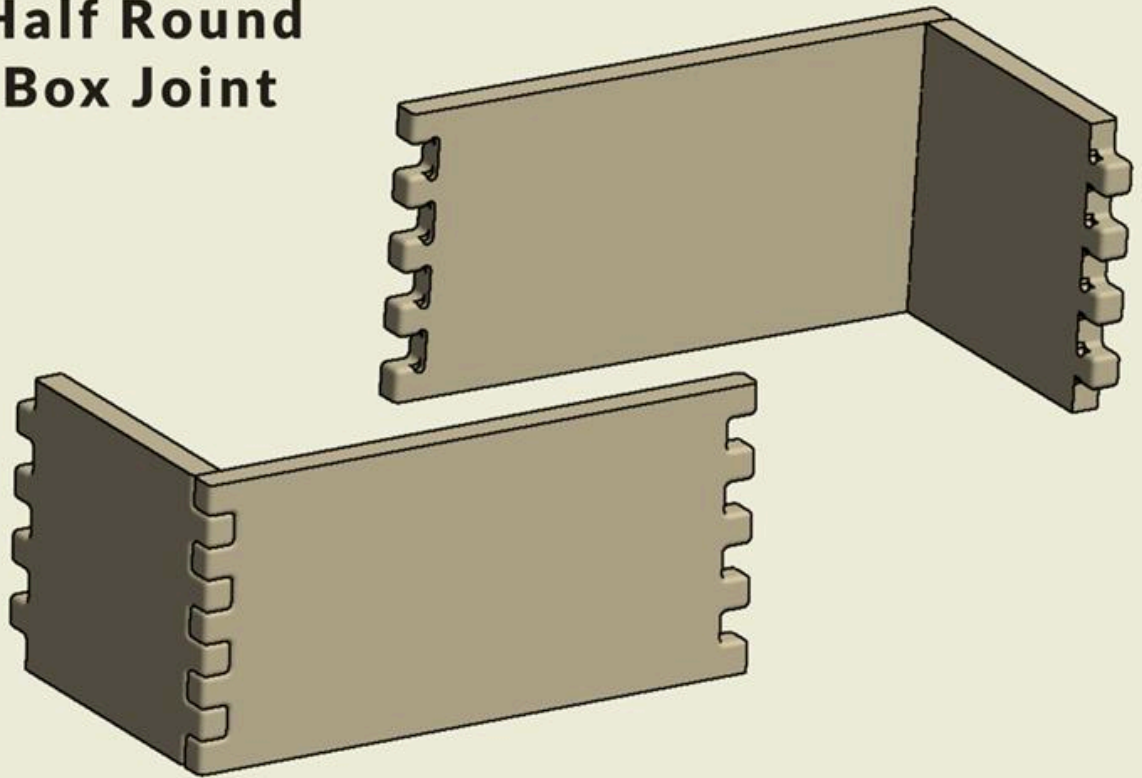
Z-depth settings for through joints should be $\text{material_thickness} + 0.010'' - 0.020''$ to cut fully through without excessive spoilboard damage. For blind joints, set exact $\text{pocket_depth} = \text{material_thickness} - \text{blind_skin}$, with blind_skin typically 2-3mm.

Conclusion

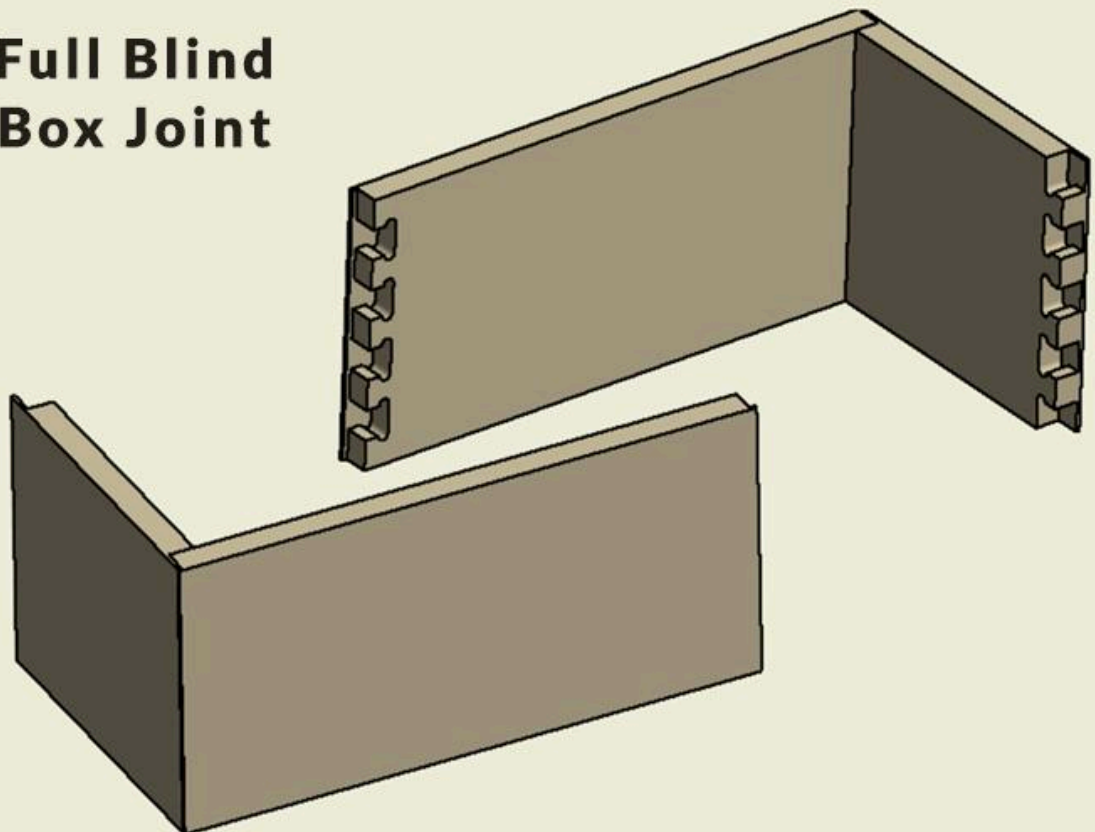
CNC box joint mathematics reduces to managing three fundamental constraints: endmill corner radius, assembly clearance, and finger count symmetry. The **half-round formula** $w = 2r$ elegantly sidesteps the corner radius problem by matching finger geometry to tool geometry. **Tolerance application of 0.010" to male fingers** (or equivalently, female socket enlargement) provides consistent, repeatable fits across material variations. The **odd finger count algorithm** $(\text{ceil}(2 \times 2 + 1))$ guarantees symmetric corner appearance regardless of board dimensions.

For parametric CAD/CAM implementation, measure actual tool and material dimensions rather than trusting nominal values, apply dogbone radius at $1.1 \times$ tool radius minimum with diagonal placement at $0.707r$ from corners, and calculate blind pocket depths as $\text{material_thickness}$ minus 2-3mm coverlet skin. These formulas, properly implemented, produce joints that assemble reliably at the first attempt.

Half Round Box Joint



Full Blind Box Joint



SAMPLE CODE WITH NO JOINERY

The following code is example SVGs for a 40"x40" frame, with no joinery applied yet. This is only provided as a "blank slate" reference to generate new methods of joinery.

```
#!/usr/bin/env python3
```

```
"""
```

McTell Parametric CNC Shadowbox Generator v10 (Clean Slate)

```
=====
```

BASED ON v9 - JOINERY STRIPPED

This version serves as the geometric baseline. All "fingers" and joinery logic have been removed. Parts are generated as pure rectangular profiles with internal features (windows, motor pockets, pilot holes) preserved.

ORIGINAL v9 HISTORY:

v3.0 - Half-Round Box Joint method.

v5.0 - SVG Arc sweep fix.

v6.0 - Canvas 65"x65" for Bezel.

v9.0 - 90-degree corner fix.

v10.0 - CLEAN SLATE: Removed all joinery/fingers to reset for new joinery logic.

```
"""
```

```
import os
```

```

import math

from pathlib import Path


#
=====
=====

# I. UNIT CONVERSION

#
=====
=====


def convert_to_mm(value, unit):

    if unit == "inches" or unit == "in":

        return value * 25.4

    elif unit == "mm":

        return value

    else:

        raise ValueError(f"Unknown unit: {unit}")


def format_inches(value_mm):

    """Format dimension in inches, removing decimals for round numbers."""

    inches = value_mm / 25.4

    if abs(inches - round(inches)) < 0.001:

        return f'{int(round(inches))}'

    else:

        return f'{inches:.2f}'

```

```
def f(val):  
    return f"{val:.4f}"
```

```
#  
=====
```

```
# II. GLOBAL PARAMETERS (EXACTLY AS v9)
```

```
#  
=====
```

```
print("=" * 70)  
  
print("McTell Parametric CNC Shadowbox Generator v10")  
  
print("STATUS: CLEAN SLATE (NO JOINERY)")  
  
print("=" * 70)
```

```
# Material & Tooling
```

```
STOCK_THICKNESS = convert_to_mm(15.0, "mm")  
  
TOOL_D_PRIMARY = convert_to_mm(0.25, "inches")  
  
TOOL_R = TOOL_D_PRIMARY / 2.0  
  
FIT_TOLERANCE = convert_to_mm(0.010, "inches")
```

```
# Finger dimensions (Preserved variables, unused in this version)
```

```
TARGET_FINGER_WIDTH = convert_to_mm(15.0, "mm")  
  
FINGER_PROUD = convert_to_mm(0.0, "mm")
```

Box Dimensions

TOTAL_WIDTH = convert_to_mm(40.0, "inches")

TOTAL_HEIGHT = convert_to_mm(40.0, "inches")

BOX_DEPTH = convert_to_mm(4.0, "inches")

Bezel & Window

BEZEL_WIDTH = convert_to_mm(1.59, "inches")

WINDOW_CUTOUT_WIDTH = TOTAL_WIDTH - (2 * BEZEL_WIDTH)

WINDOW_CUTOUT_HEIGHT = TOTAL_HEIGHT - (2 * BEZEL_WIDTH)

Back Panel Rabbet

BACK_PANEL_RABBET_WIDTH = convert_to_mm(0.625, "inches")

BACK_PANEL_RABBET_DEPTH = convert_to_mm(0.3, "inches")

Motor Pocket

MOTOR_POCKET_ENABLED = True

MOTOR_POCKET_SIZE = 42.0

MOTOR_MOUNT_PATTERN = 31.0

PHOTOGRAPH_WIDTH = TOTAL_WIDTH - (2 * STOCK_THICKNESS)

MOTOR_POCKET_X = 0.4155 * PHOTOGRAPH_WIDTH

MOTOR_CABLE_OFFSET = convert_to_mm(3.0, "inches")

CABLE_HOLE_X = MOTOR_POCKET_X - MOTOR_CABLE_OFFSET

Pilot Holes

EZ_LOK_PILOT_DIA = convert_to_mm(5.0, "mm")

Canvas

CANVAS_SIZE_STD = convert_to_mm(48.0, "inches")

CANVAS_SIZE_LARGE = convert_to_mm(65.0, "inches")

PART_ANCHOR_OFFSET = convert_to_mm(4.0, "inches")

#

=====

III. LABEL GENERATION HELPER (EXACTLY AS v9)

#

=====

def generate_label_text(part_name, width_mm, height_mm, cx, cy, rotation=0):

dim_w = format_inches(width_mm)

dim_h = format_inches(height_mm)

transform = f'transform="rotate({rotation} {f(cx)} {f(cy)})"' if rotation != 0 else "

label = f''' <text x="{f(cx)}" y="{f(cy)}" text-anchor="middle" {transform}>

<tspan x="{f(cx)}" dy="0" font-family="Josefin Sans" font-weight="600" font-size="28.22"
fill="#FF00FF">{part_name}</tspan>

```
<tspan x="{f(cx)}" dy="35" font-family="Josefin Sans" font-weight="400" font-size="21.17"
fill="#FF00FF">{dim_w} x {dim_h}</tspan>
```

```
</text>""
```

```
return label
```

```
#
```

```
=====
=====
```

```
# IV. GEOMETRY HELPERS (STRIPPED)
```

```
#
```

```
=====
=====
```

```
# `compute_finger_layout` and `draw_smart_finger` REMOVED.
```

```
# This ensures no old logic accidentally leaks in.
```

```
#
```

```
=====
=====
```

```
# V. FRONT BEZEL GENERATOR (CLEAN RECTANGLE)
```

```
#
```

```
=====
=====
```

```
def generate_front_bezel_svg():
```

```
    print("GENERATING: FRONT_BEZEL (65x65 Canvas, Clean Rectangle)")
```

```
    canvas_sz = CANVAS_SIZE_LARGE
```

```
w = TOTAL_WIDTH
```

```
h = TOTAL_HEIGHT
```

```
anchor_x = (canvas_sz - w) / 2
```

```
anchor_y = (canvas_sz - h) / 2
```

```
# Simple Rectangle Path (Clockwise)
```

```
path_cmds = []
```

```
path_cmds.append(f"M {f(anchor_x)} {f(anchor_y)}") # Top Left
```

```
path_cmds.append(f"L {f(anchor_x + w)} {f(anchor_y)}") # Top Right
```

```
path_cmds.append(f"L {f(anchor_x + w)} {f(anchor_y + h)}") # Bottom Right
```

```
path_cmds.append(f"L {f(anchor_x)} {f(anchor_y + h)}") # Bottom Left
```

```
path_cmds.append("Z")
```

```
# Window Cutout (Preserved exactly from v9)
```

```
win_x = anchor_x + BEZEL_WIDTH
```

```
win_y = anchor_y + BEZEL_WIDTH
```

```
win_w = WINDOW_CUTOUT_WIDTH
```

```
win_h = WINDOW_CUTOUT_HEIGHT
```

```
corner_r = TOOL_R
```

```
window_path = (
```

```
    f"M {f(win_x + corner_r)} {f(win_y)} "
```

```
    f"L {f(win_x + win_w - corner_r)} {f(win_y)} "
```

```

f"A {f(corner_r)} {f(corner_r)} 0 0 1 {f(win_x + win_w)} {f(win_y + corner_r)} "
f"L {f(win_x + win_w)} {f(win_y + win_h - corner_r)} "
f"A {f(corner_r)} {f(corner_r)} 0 0 1 {f(win_x + win_w - corner_r)} {f(win_y + win_h)} "
f"L {f(win_x + corner_r)} {f(win_y + win_h)} "
f"A {f(corner_r)} {f(corner_r)} 0 0 1 {f(win_x)} {f(win_y + win_h - corner_r)} "
f"L {f(win_x)} {f(win_y + corner_r)} "
f"A {f(corner_r)} {f(corner_r)} 0 0 1 {f(win_x + corner_r)} {f(win_y)} Z"
)

```

```

label_text = generate_label_text("FRONT_BEZEL", w, h, anchor_x + w/2, anchor_y + h/2)

```

```

svg = f'<?xml version="1.0" encoding="UTF-8"?>

<svg xmlns="http://www.w3.org/2000/svg" width="{f(canvas_sz)}mm"
height="{f(canvas_sz)}mm" viewBox="0 0 {f(canvas_sz)} {f(canvas_sz)}">

<title>FRONT_BEZEL</title>

{label_text}

<path d="{ ' '.join(path_cmds)}" fill="none" stroke="#000000" stroke-width="0.5"/>

<path d="{window_path}" fill="none" stroke="#FF0000" stroke-width="0.5"/>

</svg>'

return svg

```

```

#
=====
=====

```

```

# VI. RAIL GENERATOR (CLEAN RECTANGLE)

```

```
#
```

```
=====
```

```
def generate_rail_svg(rail_name, length, is_horizontal, has_motor_pocket=False, rotation_deg=0, label_offset=(0,0)):
```

```
    print(f"GENERATING: {rail_name} ({rotation_deg} deg, Clean Rectangle)")
```

```
    rail_w = length
```

```
    rail_h = BOX_DEPTH
```

```
    canvas_sz = CANVAS_SIZE_STD
```

```
    anchor_x = (canvas_sz - rail_w) / 2
```

```
    anchor_y = (canvas_sz - rail_h) / 2
```

```
    # Simple Rectangle Path (Clockwise)
```

```
    path_cmds = []
```

```
    path_cmds.append(f"M {f(anchor_x)} {f(anchor_y)}")    # Top Left
```

```
    path_cmds.append(f"L {f(anchor_x + rail_w)} {f(anchor_y)}")    # Top Right
```

```
    path_cmds.append(f"L {f(anchor_x + rail_w)} {f(anchor_y + rail_h)}")    # Bottom Right
```

```
    path_cmds.append(f"L {f(anchor_x)} {f(anchor_y + rail_h)}")    # Bottom Left
```

```
    path_cmds.append("Z")
```

```
    # Motor Pocket (Preserved exactly from v9)
```

```
    motor_elements = ""
```

```
    if has_motor_pocket and MOTOR_POCKET_ENABLED:
```

```
px = anchor_x + MOTOR_POCKET_X
```

```
py = anchor_y + (rail_h / 2)
```

```
sz = MOTOR_POCKET_SIZE
```

```
motor_elements = f'''
```

```
<rect x="{f(px - sz/2)}" y="{f(py - sz/2)}" width="{f(sz)}" height="{f(sz)}" fill="#FFFFFF"
stroke="#0000FF" stroke-width="0.5"/>
```

```
<circle cx="{f(px)}" cy="{f(py)}" r="{f(6.35)}" fill="#FFFFFF" stroke="#FF0000"
stroke-width="0.5"/>
```

```
<circle cx="{f(px - MOTOR_MOUNT_PATTERN/2)}" cy="{f(py -
MOTOR_MOUNT_PATTERN/2)}" r="1.7" fill="#FFFFFF" stroke="#FF0000"
stroke-width="0.5"/>
```

```
<circle cx="{f(px + MOTOR_MOUNT_PATTERN/2)}" cy="{f(py -
MOTOR_MOUNT_PATTERN/2)}" r="1.7" fill="#FFFFFF" stroke="#FF0000"
stroke-width="0.5"/>
```

```
<circle cx="{f(px - MOTOR_MOUNT_PATTERN/2)}" cy="{f(py +
MOTOR_MOUNT_PATTERN/2)}" r="1.7" fill="#FFFFFF" stroke="#FF0000"
stroke-width="0.5"/>
```

```
<circle cx="{f(px + MOTOR_MOUNT_PATTERN/2)}" cy="{f(py +
MOTOR_MOUNT_PATTERN/2)}" r="1.7" fill="#FFFFFF" stroke="#FF0000"
stroke-width="0.5"/>
```

```
<circle cx="{f(anchor_x + CABLE_HOLE_X)}" cy="{f(py)}" r="{f(6.35)}" fill="#FFFFFF"
stroke="#FF0000" stroke-width="0.5"/>'''
```

```
canvas_cx = canvas_sz / 2
```

```
canvas_cy = canvas_sz / 2
```

```
label_cx = canvas_cx + label_offset[0]
```

```
label_cy = canvas_cy + label_offset[1]
```

```
    transform_attr = f'transform="rotate({rotation_deg} {f(canvas_cx)} {f(canvas_cy)})"' if
rotation_deg != 0 else "
```

```
    svg = f'<?xml version="1.0" encoding="UTF-8"?>

<svg xmlns="http://www.w3.org/2000/svg" width="{f(canvas_sz)}mm"
height="{f(canvas_sz)}mm" viewBox="0 0 {f(canvas_sz)} {f(canvas_sz)}">

<title>{rail_name}</title>

{generate_label_text(rail_name, rail_w, rail_h, label_cx, label_cy, rotation_deg)}

<g {transform_attr}>

    <path d="{ ' '.join(path_cmds)}" fill="#FF97F5" stroke="#000000" stroke-width="0.3528"/>

    {motor_elements}

</g>

</svg>'

    return svg
```

```
#
=====
=====
```

```
# VII. BACK PANEL (EXACTLY AS v9)
```

```
#
=====
=====
```

```
def generate_back_panel_svg():

    print("GENERATING: BACK_PANEL (Unchanged)")

    w = TOTAL_WIDTH
```

h = TOTAL_HEIGHT

```
outer_path = f"M {f(PART_ANCHOR_OFFSET)} {f(PART_ANCHOR_OFFSET)} L  
{f(PART_ANCHOR_OFFSET+w)} {f(PART_ANCHOR_OFFSET)} L {f(PART_ANCHOR_OFFSET+w)}  
{f(PART_ANCHOR_OFFSET+h)} L {f(PART_ANCHOR_OFFSET)} {f(PART_ANCHOR_OFFSET+h)}  
Z"
```

rw = BACK_PANEL_RABBET_WIDTH

```
rabbit_path = f"M {f(PART_ANCHOR_OFFSET+rw)} {f(PART_ANCHOR_OFFSET+rw)} L  
{f(PART_ANCHOR_OFFSET+w-rw)} {f(PART_ANCHOR_OFFSET+rw)} L  
{f(PART_ANCHOR_OFFSET+w-rw)} {f(PART_ANCHOR_OFFSET+h-rw)} L  
{f(PART_ANCHOR_OFFSET+rw)} {f(PART_ANCHOR_OFFSET+h-rw)} Z"
```

hole_offset = BACK_PANEL_RABBET_WIDTH / 2

hole_r = EZ_LOK_PILOT_DIA / 2

holes = []

```
holes.append((PART_ANCHOR_OFFSET + hole_offset, PART_ANCHOR_OFFSET +  
hole_offset))
```

```
holes.append((PART_ANCHOR_OFFSET + w - hole_offset, PART_ANCHOR_OFFSET +  
hole_offset))
```

```
holes.append((PART_ANCHOR_OFFSET + hole_offset, PART_ANCHOR_OFFSET + h -  
hole_offset))
```

```
holes.append((PART_ANCHOR_OFFSET + w - hole_offset, PART_ANCHOR_OFFSET + h -  
hole_offset))
```

```
holes.append((PART_ANCHOR_OFFSET + w/3, PART_ANCHOR_OFFSET + hole_offset))
```

```
holes.append((PART_ANCHOR_OFFSET + 2*w/3, PART_ANCHOR_OFFSET + hole_offset))
```

```
holes.append((PART_ANCHOR_OFFSET + w/3, PART_ANCHOR_OFFSET + h - hole_offset))
```

```
holes.append((PART_ANCHOR_OFFSET + 2*w/3, PART_ANCHOR_OFFSET + h - hole_offset))
```

```

holes.append((PART_ANCHOR_OFFSET + hole_offset, PART_ANCHOR_OFFSET + h/3))
holes.append((PART_ANCHOR_OFFSET + hole_offset, PART_ANCHOR_OFFSET + 2*h/3))
holes.append((PART_ANCHOR_OFFSET + w - hole_offset, PART_ANCHOR_OFFSET + h/3))
holes.append((PART_ANCHOR_OFFSET + w - hole_offset, PART_ANCHOR_OFFSET + 2*h/3))

hole_elements = "\n".join([

    f' <circle cx="{f(hx)}" cy="{f(hy)}" r="{f(hole_r)}" fill="#FFFFFF" stroke="#FF0000"
stroke-width="0.5"/>'

    for hx, hy in holes

])

label_text = generate_label_text("BACK_PANEL", w, h, PART_ANCHOR_OFFSET + w/2,
PART_ANCHOR_OFFSET + h/2)

return f'''<?xml version="1.0" encoding="UTF-8"?>

<svg xmlns="http://www.w3.org/2000/svg" width="{f(CANVAS_SIZE_STD)}mm"
height="{f(CANVAS_SIZE_STD)}mm" viewBox="0 0 {f(CANVAS_SIZE_STD)}
{f(CANVAS_SIZE_STD)}">

<title>BACK_PANEL</title>

{label_text}

<path d="{f(outer_path)}" fill="none" stroke="#000000" stroke-width="0.5"/>

<path d="{f(rabbet_path)}" fill="none" stroke="#0000FF" stroke-width="0.5"/>

{hole_elements}

</svg>'''

```

```
#
```

```
=====
```

```
# VIII. MAIN EXECUTION
```

```
#
```

```
=====
```

```
def main():
```

```
    # Updated Output Directory to v10
```

```
    output_dir = Path("/Users/joelsilverman/Desktop/2026 Files/26-005 Gemini-created CNC Box  
Creator/Gemini McTell CNC Plans/McTell Gemini-Generated SVGs v10")
```

```
    output_dir.mkdir(parents=True, exist_ok=True)
```

```
    OFFSET_5_INCH = convert_to_mm(5.0, "inches")
```

```
    files = {
```

```
        "FRONT_BEZEL_v10.svg": generate_front_bezel_svg(),
```

```
        "TOP_RAIL_v10.svg": generate_rail_svg("TOP_RAIL", TOTAL_WIDTH, is_horizontal=True,  
rotation_deg=0, label_offset=(0, -OFFSET_5_INCH)),
```

```
        "BOTTOM_RAIL_v10.svg": generate_rail_svg("BOTTOM_RAIL", TOTAL_WIDTH,  
is_horizontal=True, has_motor_pocket=True, rotation_deg=180, label_offset=(0,  
OFFSET_5_INCH)),
```

```
        "LEFT_RAIL_v10.svg": generate_rail_svg("LEFT_RAIL", TOTAL_HEIGHT, is_horizontal=False,  
rotation_deg=-90, label_offset=(-OFFSET_5_INCH, 0)),
```

```
        "RIGHT_RAIL_v10.svg": generate_rail_svg("RIGHT_RAIL", TOTAL_HEIGHT,  
is_horizontal=False, rotation_deg=90, label_offset=(OFFSET_5_INCH, 0)),
```

```
        "BACK_PANEL_v10.svg": generate_back_panel_svg(),
```

```
}
```

```
for filename, content in files.items():
```

```
    with open(output_dir / filename, 'w') as fh:
```

```
        fh.write(content)
```

```
    print(f" OK {filename}")
```

```
print(f"\nSUCCESS. Files saved to: {output_dir}")
```

```
if __name__ == "__main__":
```

```
    main()
```

3D BLENDER SIMULATION

```
import bpy

import bmesh

import math

import os

# =====

# MCTELL SHADOWBOX - FULL ASSEMBLY v40

# - Feature: Robust Back Panel Generation & Texturing

# - Texture Path: Hardcoded to user spec

# =====

def delete_all():

    bpy.ops.object.select_all(action='SELECT')

    bpy.ops.object.delete(use_global=False)

def mm_to_in(mm): return mm / 25.4

# --- PARAMETERS ---

TOTAL_WIDTH_IN = 40.0

TOTAL_HEIGHT_IN = 40.0

BOX_DEPTH_IN = 4.0

STOCK_THICKNESS_MM = 15.0

STOCK_IN = mm_to_in(STOCK_THICKNESS_MM)

BEZEL_WIDTH_IN = 1.59

WINDOW_W_IN = TOTAL_WIDTH_IN - (2 * BEZEL_WIDTH_IN)

WINDOW_H_IN = TOTAL_HEIGHT_IN - (2 * BEZEL_WIDTH_IN)
```

```
# BACK PANEL RABBET SPECS
```

```
BACK_RABBET_W_IN = 0.625
```

```
BACK_RABBET_D_IN = 0.3
```

```
SCREW_PILOT_DIA_MM = 5.2
```

```
SCREW_PILOT_DIA_IN = mm_to_in(SCREW_PILOT_DIA_MM)
```

```
# IMAGE PATH
```

```
TEXTURE_PATH = "/Users/joelsilverman/Desktop/2025 Files/25-009 Blind Willie McTell  
Rephotograph/Blender for Animation/McTell Lidargraph 44 low res mockup only.jpg"
```

```
# JOINERY CONSTANTS
```

```
TARGET_FINGER_W_MM = 15.0
```

```
TARGET_FINGER_W_IN = mm_to_in(TARGET_FINGER_W_MM)
```

```
BLIND_SKIN_MM = 3.0
```

```
BLIND_SKIN_IN = mm_to_in(BLIND_SKIN_MM)
```

```
FIT_TOLERANCE_IN = 0.010
```

```
TOOL_DIA_IN = 0.25
```

```
TOOL_R_IN = TOOL_DIA_IN / 2.0
```

```
POCKET_DEPTH_IN = STOCK_IN - BLIND_SKIN_IN
```

```
FINGER_LEN_IN = POCKET_DEPTH_IN - 0.01
```

```
# =====
```

```
# HELPER FUNCTIONS
```

```
# =====
```

```
def create_cube(name, w, h, d, x, y, z):
```

```
    # Create cube centered at origin
```

```
    bpy.ops.mesh.primitive_cube_add(size=1, location=(0, 0, 0))
```

```

obj = bpy.context.active_object

obj.name = name

# Scale to dimensions

obj.dimensions = (w, h, d)

# Apply Scale so dimensions are real geometry

bpy.ops.object.transform_apply(location=False, rotation=False, scale=True)

# Move to desired location

obj.location = (x, y, z)

return obj

```

```

def compute_finger_layout(edge_len, target_w):

    count = round(edge_len / target_w)

    if count % 2 == 0: count += 1

    return count, edge_len / count

```

```

def add_t_bones(cutters_list, cx, cy, cz, depth, axis='Z'):

    rot = (0, 0, 0)

    if axis == 'X': rot = (0, math.radians(90), 0)

    elif axis == 'Y': rot = (math.radians(90), 0, 0)

    bpy.ops.mesh.primitive_cylinder_add(radius=TOOL_R_IN, depth=depth, location=(cx, cy, cz),
rotation=rot)

    cutters_list.append(bpy.context.active_object)

```

```

def perform_boolean(target, tools, operation='DIFFERENCE'):

    if not tools: return

    bpy.ops.object.select_all(action='DESELECT')

    # Clean up tools list (remove deleted/None)

    valid_tools = [t for t in tools if t is not None]

```

```
if not valid_tools: return
```

```
# Join tools into one master cutter
```

```
if len(valid_tools) > 1:
```

```
    for t in valid_tools: t.select_set(True)
```

```
    bpy.context.view_layer.objects.active = valid_tools[0]
```

```
    bpy.ops.object.join()
```

```
master = valid_tools[0]
```

```
bpy.ops.object.select_all(action='DESELECT')
```

```
target.select_set(True)
```

```
bpy.context.view_layer.objects.active = target
```

```
bpy.ops.object.modifier_add(type='BOOLEAN')
```

```
target.modifiers["Boolean"].object = master
```

```
target.modifiers["Boolean"].operation = operation
```

```
target.modifiers["Boolean"].solver = 'EXACT'
```

```
bpy.ops.object.modifier_apply(modifier="Boolean")
```

```
bpy.ops.object.select_all(action='DESELECT')
```

```
master.select_set(True)
```

```
bpy.ops.object.delete()
```

```
# =====
```

```
# 1. FRONT BEZEL
```

```
# =====
```

```
def create_front_bezel():
```

```
    # Centered at 0,0,0
```

```

bezel = create_cube("FRONT_BEZEL", TOTAL_WIDTH_IN, TOTAL_HEIGHT_IN, STOCK_IN, 0, 0,
STOCK_IN/2)

# Square Window Cutout

window = create_cube("Window_Cut", WINDOW_W_IN, WINDOW_H_IN, STOCK_IN*3, 0, 0,
STOCK_IN/2)

cutters = [window]

# Horizontal Sockets

count_h, width_h = compute_finger_layout(TOTAL_WIDTH_IN, TARGET_FINGER_W_IN)

start_x = -TOTAL_WIDTH_IN / 2

for y_edge in [-TOTAL_HEIGHT_IN/2, TOTAL_HEIGHT_IN/2]:

    is_top = (y_edge > 0)

    cy = y_edge - (STOCK_IN/2) if is_top else y_edge + (STOCK_IN/2)

    inner_y = y_edge - STOCK_IN if is_top else y_edge + STOCK_IN

    for i in range(count_h):

        if i % 2 == 1:

            cx = start_x + (i * width_h) + (width_h/2)

            c = create_cube(f"Sock_H_{i}", width_h, STOCK_IN, POCKET_DEPTH_IN, cx, cy,
POCKET_DEPTH_IN/2)

            cutters.append(c)

            add_t_bones(cutters, cx - width_h/2, inner_y, POCKET_DEPTH_IN/2,
POCKET_DEPTH_IN)

            add_t_bones(cutters, cx + width_h/2, inner_y, POCKET_DEPTH_IN/2,
POCKET_DEPTH_IN)

# Vertical Sockets

count_v, width_v = compute_finger_layout(TOTAL_HEIGHT_IN, TARGET_FINGER_W_IN)

start_y = -TOTAL_HEIGHT_IN / 2

for x_edge in [-TOTAL_WIDTH_IN/2, TOTAL_WIDTH_IN/2]:

    is_right = (x_edge > 0)

```

```

    cx = x_edge - (STOCK_IN/2) if is_right else x_edge + (STOCK_IN/2)

    inner_x = x_edge - STOCK_IN if is_right else x_edge + STOCK_IN

    for i in range(count_v):

        if i % 2 == 1:

            cy = start_y + (i * width_v) + (width_v/2)

            c = create_cube(f"Sock_V_{i}", STOCK_IN, width_v, POCKET_DEPTH_IN, cx, cy,
POCKET_DEPTH_IN/2)

            cutters.append(c)

            add_t_bones(cutters, inner_x, cy - width_v/2, POCKET_DEPTH_IN/2,
POCKET_DEPTH_IN)

            add_t_bones(cutters, inner_x, cy + width_v/2, POCKET_DEPTH_IN/2,
POCKET_DEPTH_IN)

    perform_boolean(bezel, cutters)

    return bezel

```

```

# =====

```

```

# 2. HORIZONTAL RAILS

```

```

# =====

```

```

def create_horizontal_rail(name):

    rail = create_cube(name, TOTAL_WIDTH_IN, BOX_DEPTH_IN, STOCK_IN, 0, 0, STOCK_IN/2)

    cutters = []

    count_h, width_h = compute_finger_layout(TOTAL_WIDTH_IN, TARGET_FINGER_W_IN)

    start_x = -TOTAL_WIDTH_IN / 2

    face_edge_y = BOX_DEPTH_IN / 2

    for i in range(count_h):

        if i % 2 == 0:

            cx = start_x + (i * width_h) + (width_h/2)

            cy = face_edge_y - (FINGER_LEN_IN/2)

```

```

        c = create_cube(f"FaceGap_{i}", width_h, FINGER_LEN_IN, STOCK_IN*2, cx, cy,
STOCK_IN/2)

        cutters.append(c)

count_d, width_d = compute_finger_layout(BOX_DEPTH_IN, TARGET_FINGER_W_IN)

start_y = -BOX_DEPTH_IN / 2

for x_edge in [-TOTAL_WIDTH_IN/2, TOTAL_WIDTH_IN/2]:

    is_right = (x_edge > 0)

    for i in range(count_d):

        if i % 2 == 0:

            cy = start_y + (i * width_d) + (width_d/2)

            cx = x_edge - (FINGER_LEN_IN/2) if is_right else x_edge + (FINGER_LEN_IN/2)

            c = create_cube(f"EndGap_{i}", FINGER_LEN_IN, width_d, STOCK_IN*2, cx, cy,
STOCK_IN/2)

            cutters.append(c)

perform_boolean(rail, cutters)

return rail

# =====

# 3. VERTICAL RAILS

# =====

def create_vertical_rail(name):

    rail = create_cube(name, TOTAL_HEIGHT_IN, BOX_DEPTH_IN, STOCK_IN, 0, 0, STOCK_IN/2)

    cutters = []

count_v, width_v = compute_finger_layout(TOTAL_HEIGHT_IN, TARGET_FINGER_W_IN)

start_x = -TOTAL_HEIGHT_IN / 2

face_edge_y = BOX_DEPTH_IN / 2

for i in range(count_v):

```

```

        if i % 2 == 0:

            cx = start_x + (i * width_v) + (width_v/2)

            cy = face_edge_y - (FINGER_LEN_IN/2)

            c = create_cube(f"FaceGap_{i}", width_v, FINGER_LEN_IN, STOCK_IN*2, cx, cy,
STOCK_IN/2)

            cutters.append(c)


count_d, width_d = compute_finger_layout(BOX_DEPTH_IN, TARGET_FINGER_W_IN)

start_y = -BOX_DEPTH_IN / 2

for x_edge in [-TOTAL_HEIGHT_IN/2, TOTAL_HEIGHT_IN/2]:

    is_top_end = (x_edge > 0)

    for i in range(count_d):

        if i % 2 == 1:

            cy = start_y + (i * width_d) + (width_d/2)

            cx = x_edge - (STOCK_IN/2) if is_top_end else x_edge + (STOCK_IN/2)

            c = create_cube(f"EndSock_{i}", STOCK_IN, width_d, POCKET_DEPTH_IN, cx, cy,
STOCK_IN - POCKET_DEPTH_IN/2)

            cutters.append(c)

            inner_x = x_edge - STOCK_IN if is_top_end else x_edge + STOCK_IN

            add_t_bones(cutters, inner_x, cy - width_d/2, STOCK_IN - POCKET_DEPTH_IN/2,
POCKET_DEPTH_IN)

            add_t_bones(cutters, inner_x, cy + width_d/2, STOCK_IN - POCKET_DEPTH_IN/2,
POCKET_DEPTH_IN)


perform_boolean(rail, cutters)

return rail

```

```

# =====

```

```

# 4. BACK PANEL (FIXED)

```

```

# =====

```

```

def create_back_panel():

```

```

# 1. Create Base Slab

# Z-Up. Bottom at 0, Top at STOCK_IN.

panel = create_cube("BACK_PANEL", TOTAL_WIDTH_IN, TOTAL_HEIGHT_IN, STOCK_IN, 0, 0,
STOCK_IN/2)

# 2. Rabbet Geometry

# The Rabbet cuts into the perimeter of the 'Top' Face (Z=STOCK)

# leaving a "Tongue" in the center.

# The Edge (Flange) will be thin (Stock - RabbetDepth).

# The Center will be full thickness.

rw = BACK_RABBET_W_IN
rd = BACK_RABBET_D_IN

cutters = []

# Dimensions for cutters:

# We want to remove the perimeter.

# Left Edge Cutter

c_l = create_cube("C_L", rw, TOTAL_HEIGHT_IN, rd*2,
-TOTAL_WIDTH_IN/2 + rw/2, 0, STOCK_IN) # Centered on edge

cutters.append(c_l)

# Right Edge Cutter

c_r = create_cube("C_R", rw, TOTAL_HEIGHT_IN, rd*2,
TOTAL_WIDTH_IN/2 - rw/2, 0, STOCK_IN)

cutters.append(c_r)

# Top Edge Cutter (Width is total - 2*rw to avoid overlap or just let them overlap)

# Overlap is fine for boolean.

```

```

c_t = create_cube("C_T", TOTAL_WIDTH_IN, rw, rd*2,
                  0, TOTAL_HEIGHT_IN/2 - rw/2, STOCK_IN)

cutters.append(c_t)

# Bottom Edge Cutter

c_b = create_cube("C_B", TOTAL_WIDTH_IN, rw, rd*2,
                  0, -TOTAL_HEIGHT_IN/2 + rw/2, STOCK_IN)

cutters.append(c_b)

# Pilot Holes (12 holes) through the rabbet flange

hole_inset = rw / 2.0

# Coordinates relative to center

x_outer = TOTAL_WIDTH_IN/2 - hole_inset
y_outer = TOTAL_HEIGHT_IN/2 - hole_inset

# Corners

coords = [
    (-x_outer, -y_outer), (x_outer, -y_outer),
    (-x_outer, y_outer), (x_outer, y_outer)
]

# Midpoints (Simplified for 3 per side logic from doc)

# Top/Bot Mids

x_third = TOTAL_WIDTH_IN / 6 # 1/3 from center? No, 1/3 of total width.

# Total width W. Mids at W/3 and 2W/3.

# Center is 0. Left is -W/2.

# -W/2 + W/3 = -W/6. -W/2 + 2W/3 = +W/6.

coords.append((-TOTAL_WIDTH_IN/6, y_outer)) # Top Left-Mid
coords.append((TOTAL_WIDTH_IN/6, y_outer)) # Top Right-Mid
coords.append((-TOTAL_WIDTH_IN/6, -y_outer)) # Bot Left-Mid

```

```

coords.append((TOTAL_WIDTH_IN/6, -y_outer)) # Bot Right-Mid

# Side Mids

coords.append((-x_outer, -TOTAL_HEIGHT_IN/6))

coords.append((-x_outer, TOTAL_HEIGHT_IN/6))

coords.append((x_outer, -TOTAL_HEIGHT_IN/6))

coords.append((x_outer, TOTAL_HEIGHT_IN/6))


for (hx, hy) in coords:

    c = bpy.ops.mesh.primitive_cylinder_add(radius=SCREW_PILOT_DIA_IN/2,
depth=STOCK_IN*3, location=(hx, hy, 0))

    cutters.append(bpy.context.active_object)


perform_boolean(panel, cutters)


# 3. MATERIALS & TEXTURE

# Material 0: Plywood Edge/Back

mat_base = bpy.data.materials.new(name="BackPanel_Base")

mat_base.diffuse_color = (0.7, 0.5, 0.3, 1)

panel.data.materials.append(mat_base)


# Material 1: The Photo

mat_photo = bpy.data.materials.new(name="McTell_Photo")

mat_photo.use_nodes = True

nodes = mat_photo.node_tree.nodes

links = mat_photo.node_tree.links

bsdf = nodes.get("Principled BSDF")


# Texture Node

tex_image = nodes.new('ShaderNodeTexImage')

```

```

try:

    # LOAD IMAGE

    if os.path.exists(TEXTURE_PATH):

        img = bpy.data.images.load(TEXTURE_PATH)

        tex_image.image = img

        links.new(tex_image.outputs['Color'], bsdf.inputs['Base Color'])

        print("Texture loaded successfully.")

    else:

        print(f"ERROR: Texture file not found at {TEXTURE_PATH}")

        mat_photo.diffuse_color = (1, 0, 1, 1) # Bright Pink Error

except Exception as e:

    print(f"Texture Load Error: {e}")


panel.data.materials.append(mat_photo)


# 4. ASSIGN TEXTURE TO "INSIDE" FACE

# The "Inside" face is the raised Tongue surface (Top Z).

# Its Normal is +Z (0,0,1). It is the highest face.

bpy.context.view_layer.objects.active = panel

bpy.ops.object.mode_set(mode='EDIT')

bpy.ops.mesh.select_all(action='DESELECT')


bm = bmesh.from_edit_mesh(panel.data)

bm.faces.ensure_lookup_table()


target_face = None

# Find face with Normal +Z and highest Z position

for f in bm.faces:

    if f.normal.z > 0.9: # Pointing Up

```

```

# Check height. Tongue is at  $Z = \text{STOCK\_IN}/2 + \text{STOCK\_IN}/2 = \text{STOCK\_IN}$  ?

# Orig Cube Z=0 to Z=STOCK.

# Tongue Face is at Z approx STOCK_IN.

# Rabbet Floor is at Z approx STOCK_IN - R_D.

z_height = f.calc_center_median().z

if z_height > (STOCK_IN - 0.05):

    target_face = f

    f.select = True

    break

```

```

if target_face:

    panel.active_material_index = 1

    bpy.ops.object.material_slot_assign()

    # UV Map: Project from View (Top) or Cube Project

    # We want "Edge to Edge" on the Tongue.

    # Let's use Cube Project limited to bounds.

    bpy.ops.uv.cube_project(scale_to_bounds=True)

```

```

bpy.ops.object.mode_set(mode='OBJECT')

```

```

return panel

```

```

# =====

# 5. ASSEMBLY

# =====

```

```

delete_all()

```

```

bezel = create_front_bezel()

```

```

top = create_horizontal_rail("TOP_RAIL")

bot = create_horizontal_rail("BOTTOM_RAIL")

left = create_vertical_rail("LEFT_RAIL")

right = create_vertical_rail("RIGHT_RAIL")

back = create_back_panel()


EXPLODE = 2.0


# 1. Bezel (Front, Vertical)

bezel.rotation_euler = (math.radians(90), 0, 0)

bezel.location = (0, 0, TOTAL_HEIGHT_IN/2)


# 2. Rails (Behind Bezel)

top.rotation_euler = (0, 0, math.radians(180))

top.location = (0, STOCK_IN + EXPLODE, TOTAL_HEIGHT_IN - STOCK_IN/2)


bot.rotation_euler = (0, 0, math.radians(180))

bot.location = (0, STOCK_IN + EXPLODE, STOCK_IN/2)


left.rotation_euler = (0, math.radians(-90), math.radians(180))

left.location = (-TOTAL_WIDTH_IN/2 + STOCK_IN/2, STOCK_IN + EXPLODE, TOTAL_HEIGHT_IN/2)


right.rotation_euler = (0, math.radians(90), math.radians(180))

right.location = (TOTAL_WIDTH_IN/2 - STOCK_IN/2, STOCK_IN + EXPLODE, TOTAL_HEIGHT_IN/2)


# 3. Back Panel (Behind Rails)

# Rotate so "Inside" (Textured Tongue) faces -Y (Towards Bezel).

# Created Flat (Tongue Up +Z).

# Rotate +90 X -> Tongue faces -Y.

```

```

back.rotation_euler = (math.radians(90), 0, 0)

# Position

# Rails End at Y = STOCK_IN + EXPLODE + BOX_DEPTH.

# Back Panel Rabbet Flange sits on Rails.

# Back Panel Thickness = STOCK.

# If we align it:

back_y = STOCK_IN + EXPLODE + BOX_DEPTH_IN + EXPLODE

back.location = (0, back_y, TOTAL_HEIGHT_IN/2)

# Styling

def make_mat(name, color):

    if name in bpy.data.materials: return bpy.data.materials[name]

    m = bpy.data.materials.new(name=name)

    m.diffuse_color = color

    return m

bezel.data.materials.append(make_mat("Bezel", (0.8, 0.6, 0.4, 1)))

rail_mat = make_mat("Rails", (0.4, 0.6, 0.8, 1))

top.data.materials.append(rail_mat)

bot.data.materials.append(rail_mat)

left.data.materials.append(rail_mat)

right.data.materials.append(rail_mat)

print("McTell Shadowbox Assembly v40 Generated Successfully.")

```